

**MBA IN SOFTWARE  
ENGINEERING**

## **DESIGN PATTERNS I**

Prof. Dr. Rosana T. Vaccare Braga

**MBA** **USP**  
**ESALQ**

The trustworthiness, originality,  
and legality of the educational content presented is responsibility of the professor.

**The total or partial reproduction** of this material without authorization is prohibited.

Law No.  
9610/98

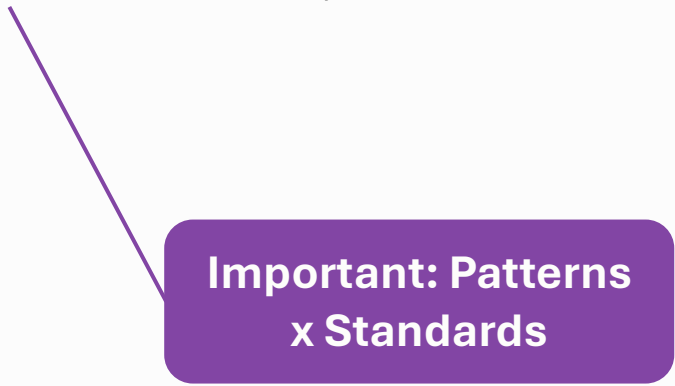
# Class organization

- Software reuse and its benefits;
- Patterns and their origin;
- Patterns vs Standards;
- Elements of a pattern, types of patterns;
- Design patterns: definitions, advantages, categories, writing formats;
- GoF patterns from the “creational” category and beginning of the “structural” category.



# Software Reuse

- **Software Reuse:** replication of any type of knowledge about a system in other similar systems, with the aim of reducing development effort and maintenance in these new systems (Biggerstaff and Perlis, 1989).
  - 70s: modules and subroutines
  - 80s: classes and application generators
  - 90s: domain analysis, components, patterns *and* frameworks;
  - 2000s: services, aspects
  - ...



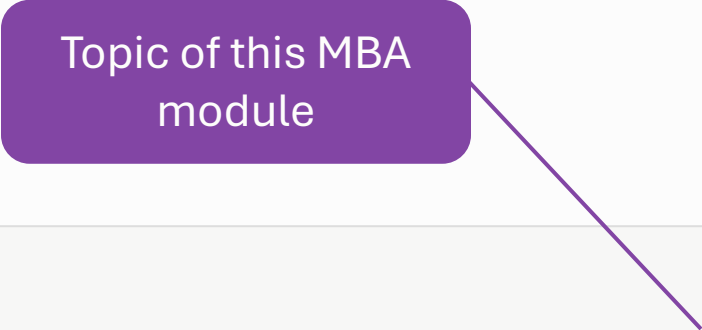
**Important: Patterns  
x Standards**

# Software Reuse

- Benefits:
  - Improved productivity rates
  - Higher quality products, more reliable, consistent, and standardized
  - Reduction of costs and time involved in software development
  - Greater flexibility in the structure of the produced software, facilitating its maintenance and evolution

# Important: Patterns x Standards

Topic of this MBA  
module



## Patterns:

- Proxy, Composite, Factory-method, etc (**design**)
- Layers, MVC, etc (architectural)
- Transaction, Item-transaction, Type-object (analysis)
- Site map, shopping cart, ... (user interaction)
- etc...

## Standards:

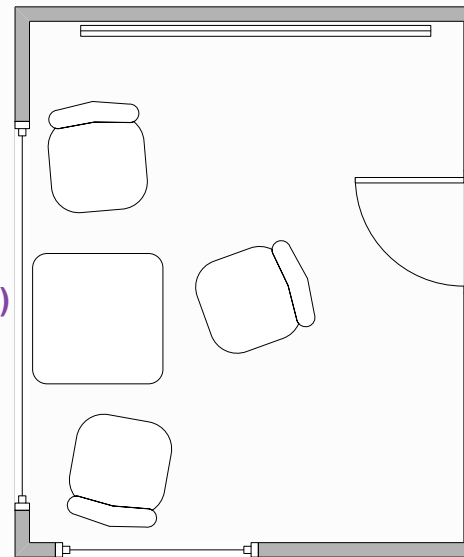
- USB, HDMI, Bluetooth, ...
- HTML, XML, ...
- ISO 9000, ISO/IEC 15504, ...
- IEEE
- etc...

Patterns: The Hillside Group - A group dedicated to design patterns. Home of the patterns library.

# Software Patterns (origin)

- Christopher Alexander (1977, 1979) - Architect
  - “Each pattern describes a problem that occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over without using it the same way twice”
  - Proposal of patterns extracted from the structures of buildings and cities from various cultures, with the aim of helping people build their communities with the best possible quality of life
  - Example of a pattern:

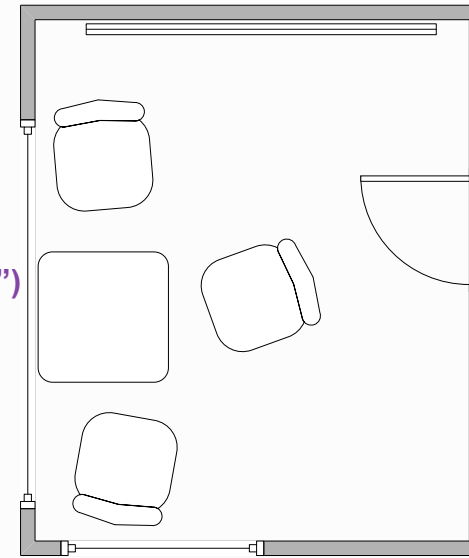
**Window place**  
("a place by the window")



\*Christopher Alexander et al, *A Pattern Language-Towns. Buildings. Construction*, 1977

# Software Patterns (origin)

**Window place**  
("a place by the window")



\*Christopher Alexander et al, *A Pattern Language- Towns.Buildings. Construction*, 1977

- Everyone loves window seats, bay windows, and large windows with low sills and comfortable chairs pulled up to them... A room without a spot like this rarely lets you feel comfortable If the room does not contain a window
- that is a "place", a person will be led to : 1. Want to
- sit and be comfortable OR 2. Be
- drawn to the light
- If comfortable places are far from the windows, there is no way to overcome this
- conflict... Therefore: in every room where you spend some time during the day, make at least one "window place"

# Software Patterns (origin)

Beck and Cunningham: 1987

small pattern language to guide  
inexperienced programmers in Smalltalk

Peter Coad: 1992

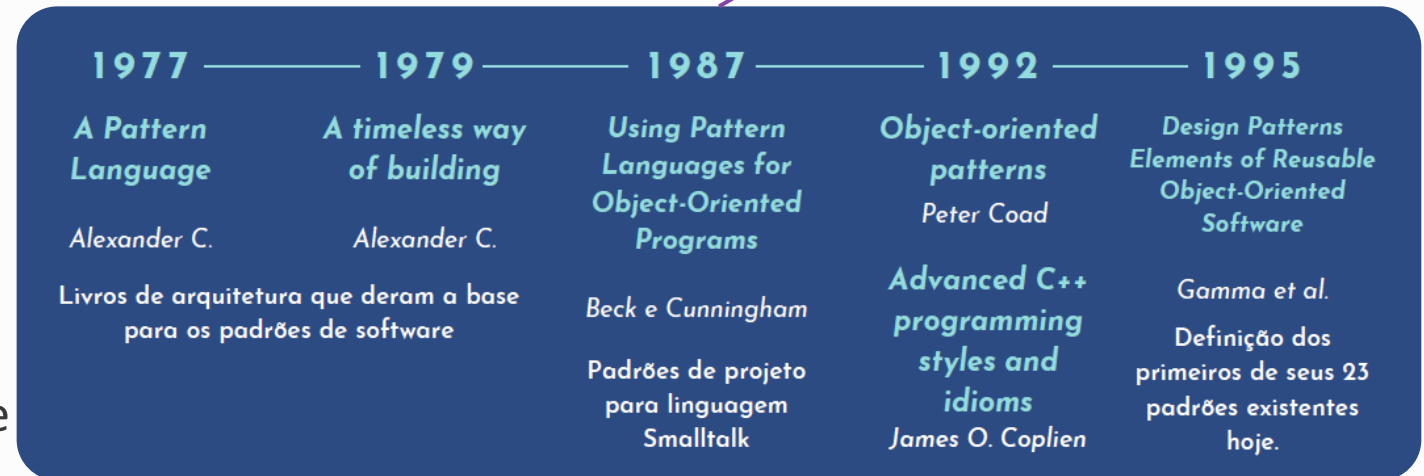
analysis patterns discovered in information  
system modeling

James Coplien – 1992

catalog of various styles ("idioms"), with the  
aim of standardizing code writing in C+

Gamma et al – 1995

In software:



**design patterns** derived from practical experience with object-oriented software development

# Why Patterns ?

- Developers accumulate solutions for problems they frequently solve
- These solutions are difficult to elaborate and can increase software productivity, quality, and uniformity
- How to document these solutions so that other, less experienced developers can use them?
- **Software Patterns:** Describe solutions to problems that frequently occur in software development (*Gamma 95*)

# A good pattern:

- solves a problem
- is a proven concept (not just theory)
- the solution is not obvious
- describes a relationship (structures and mechanisms)
- has a significant human component

# Advantages of Patterns

- **Increased productivity:** applying ready-made solutions leads to less time spent thinking about the solution, designing it, and then applying it.
- **Uniformity in software structure:** understanding software becomes easier when patterns have been used, as the patterns used throughout the software (whether in design or implementation) can be seen.
- **Immediate application by other developers:** knowing the pattern (its purpose, its structure, the context in which it should be applied, its consequences, etc.) allows it to be applied immediately even without having been the pattern's proponent.
- **Complexity reduction:** patterns come to be seen as building blocks, where the inherent complexity of each pattern is abstracted, and the whole is viewed with less complexity (divide and conquer principle).

# Categories of Patterns

- Organizational
- Architectural
- Analysis
- Design
  - **Gamma et al**
  - Others : e.g. J2EE
- Programming (styles)
- Usability
- Educational
- Etc.

Topic of this MBA module

## EXAMPLE

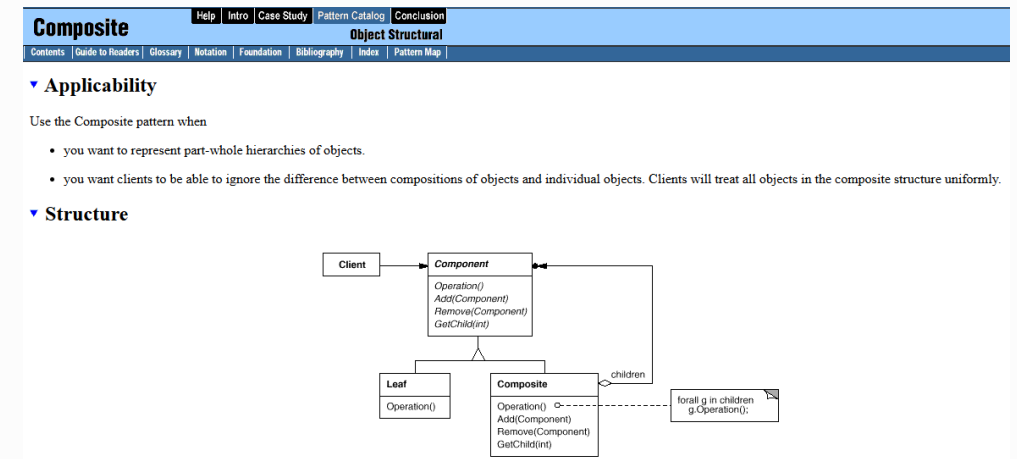
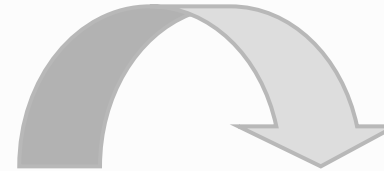
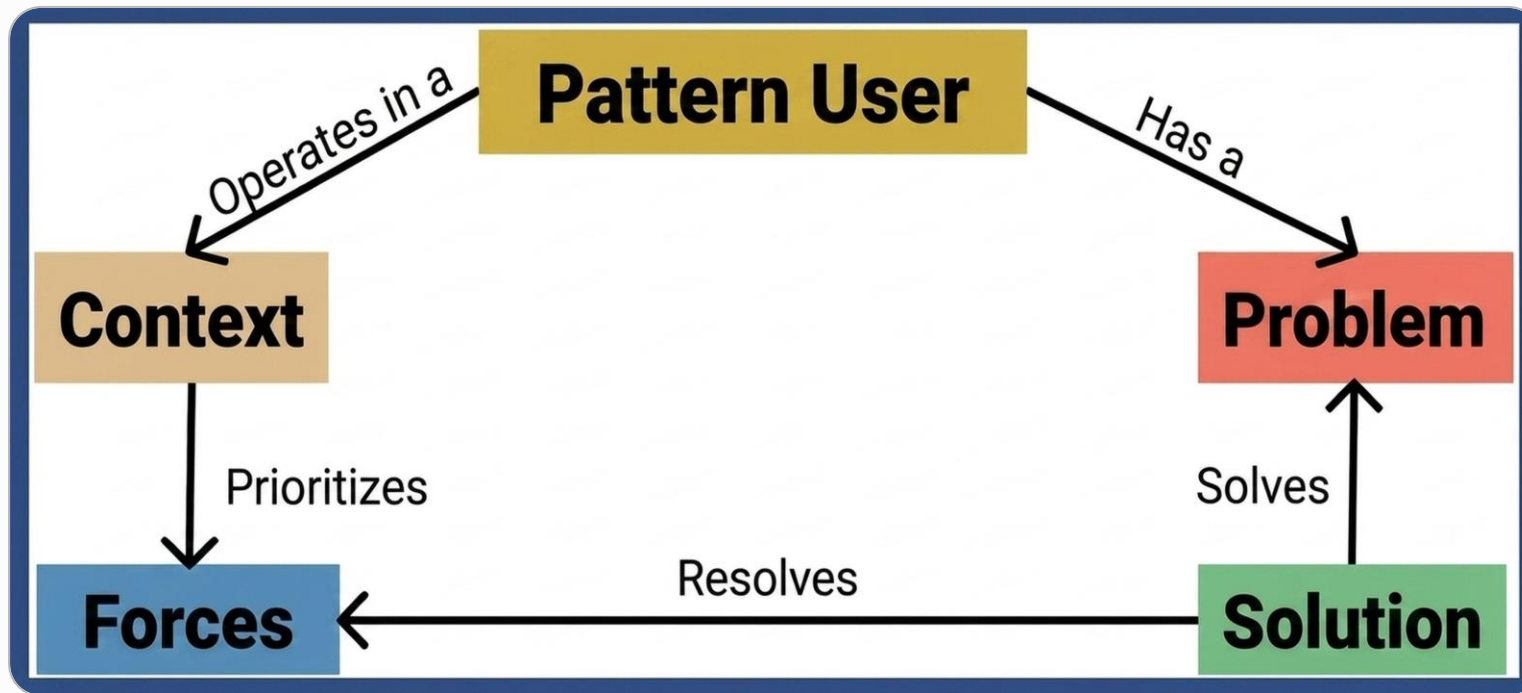


Image captured from the html version of the book published on CD (Gamma et al 1995)

# Elements of a pattern



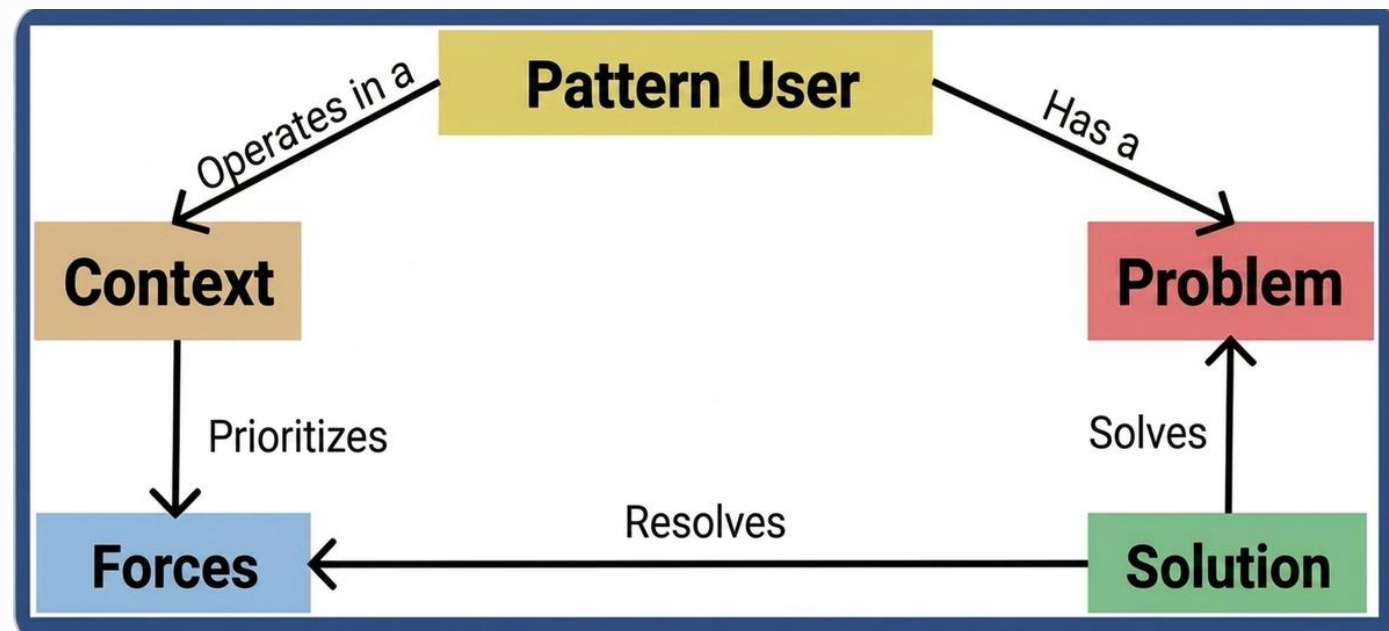
(Meszaros & Doble, 1998)

# Elements of a pattern



## Survey 1 : Problem:

*"I don't have money to take my vacation trip"*



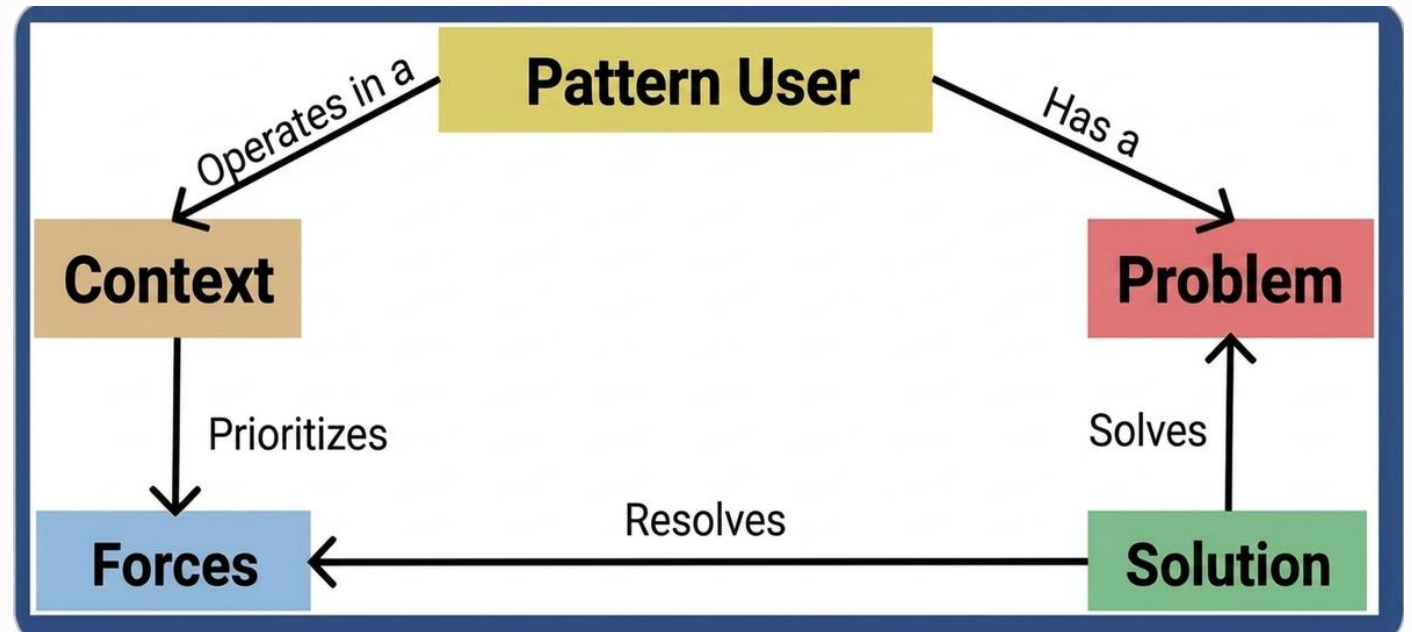
(Meszaros & Doble, 1998)

# Elements of a pattern



## Survey 2 : Problem in a given context:

*"I don't have money to take my vacation trip"*



(Meszaros & Doble, 1998)

# Format for writing software patterns

- When writing a pattern, one can combine some essential elements and other optional elements.
- Initially, one should not be tied to any writing format, and there is no obligation to use all elements in all patterns.
- As the pattern is reused, its format can be changed to facilitate its understanding and reuse.
- There are some tested and approved template suggestions: **Gamma et al. (1995), Coplien (1996) and Buschmann et al. (1996)**
  - Note: there are other formats

# Format for writing software patterns

Format used in  
this MBA  
module

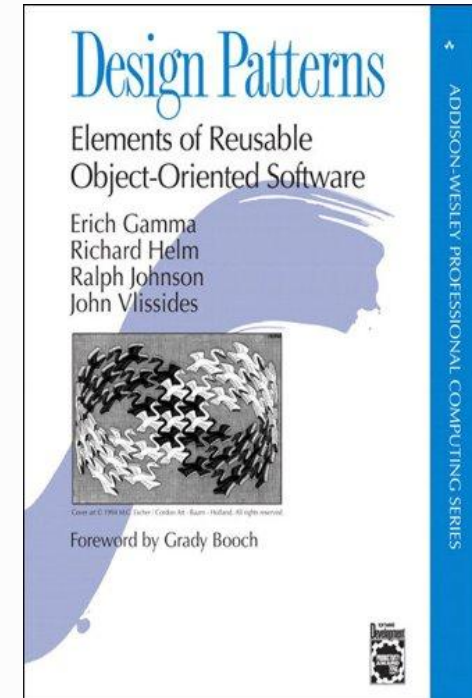
| Gamma                                                                                                                                                                                                              | Coplien                                                                                         | Buschmann                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NAME<br>Classification<br>Intent<br>Also Known As<br>Motivation<br>Applicability<br>Structure<br>Participants<br>Collaborations<br>Implementation<br>Example Code<br>Consequences<br>Known Uses<br>Pattern Related | NAME<br>CONTEXT<br>PROBLEM<br>Forces<br>Solution<br>Schema<br>CONTEXT<br>Resulting<br>Reasoning | NAME<br>Also Known As<br>EXAMPLE<br>CONTEXT<br>PROBLEM<br>Solution<br>Structure<br>Dynamics<br>Implementation<br>Solved Example<br>Variants<br>Known Uses<br>Consequences<br>See Also |

# DESIGN PATTERNS:

Describe solutions to problems that frequently occur in object-oriented software design

# Types of GoF\* Design Patterns

- Catalog of Design Patterns published in 1995 by Gamma, Helm, Johnson, and Vlissides
- Two classification criteria:
  - Purpose - reflects what the pattern does
    - Creational: deals with object creation
    - Structural: handles the composition of classes and objects
    - Behavioral: characterizes how classes and objects interact and distribute responsibilities
  - Scope in which the pattern operates:
    - Class: deals with the relationship between classes and subclasses (inheritance - static relationship)
    - Objects: deals with object manipulation (can be modified at runtime)



**\* GoF: Gang of Four – nickname given to the four authors of the book**

# GoF Design Patterns



**Survey 3:** Experience with GoF patterns



|       |        | Purpose                                                                                                                      |                                                                                                                                                                         |                                                                                                                                                                                                                             |
|-------|--------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |        | Creational                                                                                                                   | Structural                                                                                                                                                              | Behavioral                                                                                                                                                                                                                  |
| Scope | Class  | Factory Method                                                                                                               | Adapter                                                                                                                                                                 | Interpreter<br>Template Method                                                                                                                                                                                              |
|       | Object | <ul style="list-style-type: none"><li>• Abstract Factory</li><li>• Builder</li><li>• Prototype</li><li>• Singleton</li></ul> | <ul style="list-style-type: none"><li>• Adapter</li><li>• Bridge</li><li>• Composite</li><li>• Decorator</li><li>• Facade</li><li>• Flyweight</li><li>• Proxy</li></ul> | <ul style="list-style-type: none"><li>• Chain of Responsibility</li><li>• Command</li><li>• Iterator</li><li>• Mediator</li><li>• Memento</li><li>• Observer</li><li>• State</li><li>• Strategy</li><li>• Visitor</li></ul> |

# Before we begin...

- Let's understand how GoF structured the writing of the 23 design patterns?
  - **Pattern Name and Classification:** The pattern's name conveys its essence succinctly. A good name is vital because it will become part of your design vocabulary. The pattern's classification reflects the scheme presented in the previous slide.
  - **Intent:** A brief statement that answers the following questions: What does the design pattern do? What is its rationale and intent? What particular design problem or issue does it address?
  - **Also Known As:** Other known names for the pattern, if any.
  - **Motivation:** A scenario that illustrates a design problem and how the class and object structures in the pattern solve the problem. The scenario will help you understand the more abstract description of the pattern that follows.

# Before we begin...

- **Applicability:** What are the situations in which the design pattern can be applied? What are examples of bad designs that the pattern can address? How can you recognize these situations?
- **Structure:** In the case of the GoF book, a graphical representation of the classes in the pattern was adopted using a notation based on the *Object Modeling Technique (a precursor to UML (Unified Modeling Language))*. *Interaction diagrams [JCJO92, Boo94]* are also used to illustrate sequences of requests and collaborations between objects.
- **Participants:** The classes and/or objects that participate in the design pattern and their responsibilities.
- **Collaborations:** How participants collaborate to carry out their responsibilities.

# Before we begin...

- **Consequences:** How does the pattern support its objectives? What are the trade-offs and results of using the pattern? What aspect of the system's structure does it allow you to vary independently?
- **Implementation:** What pitfalls, tips, or techniques should you be aware of when implementing the pattern? Are there language-specific issues?
- **Sample Code:** Code fragments that illustrate how you can implement the pattern in an OO language.
- **Known Uses:** Examples of the pattern found in real systems. The book includes at least two examples from different domains.
- **Related Patterns:** Which design patterns are closely related to this one? What are the important differences? With which other patterns should this one be used?

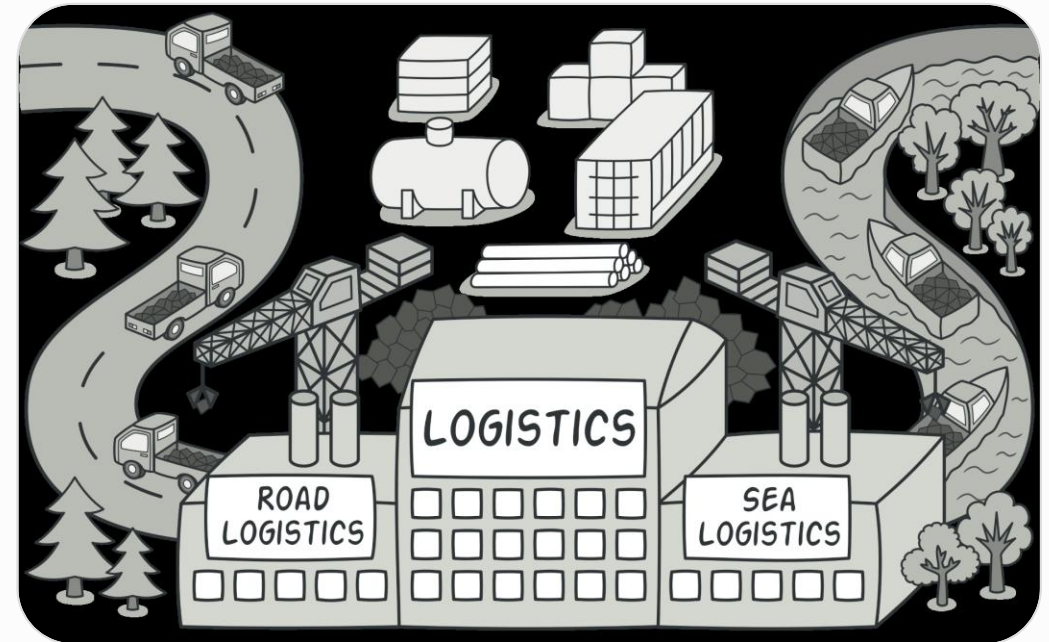
|       |        | Purpose                                                                                                                           |                                                                                                                                                                                 |                                                                                                                                                                                                                                       |
|-------|--------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |        | Creational                                                                                                                        | Structural                                                                                                                                                                      | Behavioral                                                                                                                                                                                                                            |
| Scope | Class  | Factory Method                                                                                                                    | Adapter                                                                                                                                                                         | Interpreter<br>Template Method                                                                                                                                                                                                        |
|       | Object | <ul style="list-style-type: none"> <li>• Abstract Factory</li> <li>• Builder</li> <li>• Prototype</li> <li>• Singleton</li> </ul> | <ul style="list-style-type: none"> <li>• Adapter</li> <li>• Bridge</li> <li>• Composite</li> <li>• Decorator</li> <li>• Facade</li> <li>• Flyweight</li> <li>• Proxy</li> </ul> | <ul style="list-style-type: none"> <li>• Chain of Responsibility</li> <li>• Command</li> <li>• Iterator</li> <li>• Mediator</li> <li>• Memento</li> <li>• Observer</li> <li>• State</li> <li>• Strategy</li> <li>• Visitor</li> </ul> |

# PATTERNS in the “creational” category

Responsible for the object creation process, potentially increasing code flexibility and reusability

# Factory Method Design Pattern

- Imagine you've launched a logistics management application. In the first version, you only offer truck transportation, so most of your code is contained within the Truck class
- After a while, with the success of your application, requests arrive from maritime transport companies to also incorporate maritime logistics.
  - Adding the Ship class would require changes throughout the code!
  - The code would be full of conditionals to properly handle each type of transport, although there is a lot of code that will be useful regardless of the type of transport.
- Great opportunity to use the Factory Method pattern!!!



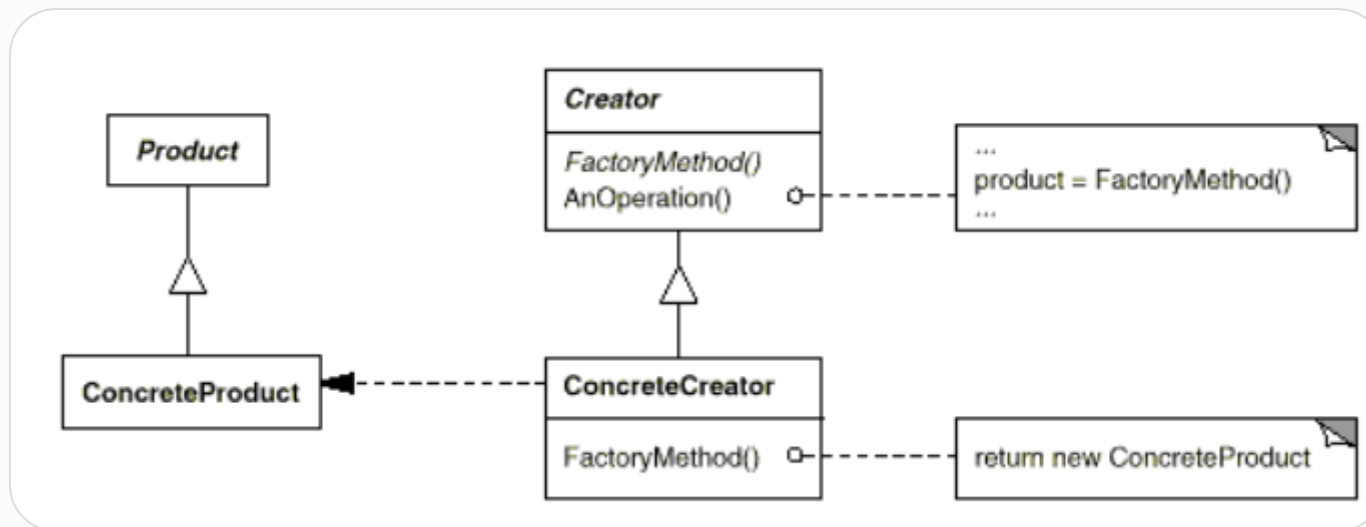
*Shvets, Alexander, 2019*

# Factory Method Design Pattern

- **Classification:** Structural Pattern/Class scope
- **Intent:** Define an interface for creating an object, but let subclasses decide which class to instantiate. The pattern allows a class to defer instantiation to subclasses.
- **Motivation:** In various types of applications, the choice of which object will be created depends on choices made at runtime, so there is no way to know which class should be instantiated. This can cause a lot of duplicated code to predict the various combinations of classes, in addition to case structures scattered throughout the code.

# Factory Method Design Pattern

- **Applicability**
- Use the Factory Method pattern when:
  - a class cannot anticipate the class of objects it must create.
  - a class wants its subclasses to specify the objects they create.
- **Structure:**



*Image captured from the html version of the book published on CD (Gamma et al 1995)*

# Factory Method Design Pattern

- **Participants:**

- Declares the factory method, which returns an object of type Product. It can also define a default implementation that returns a ConcreteProduct object.
  - It can call the factory method to create a Product object

Declares the interface of the objects that the factory method

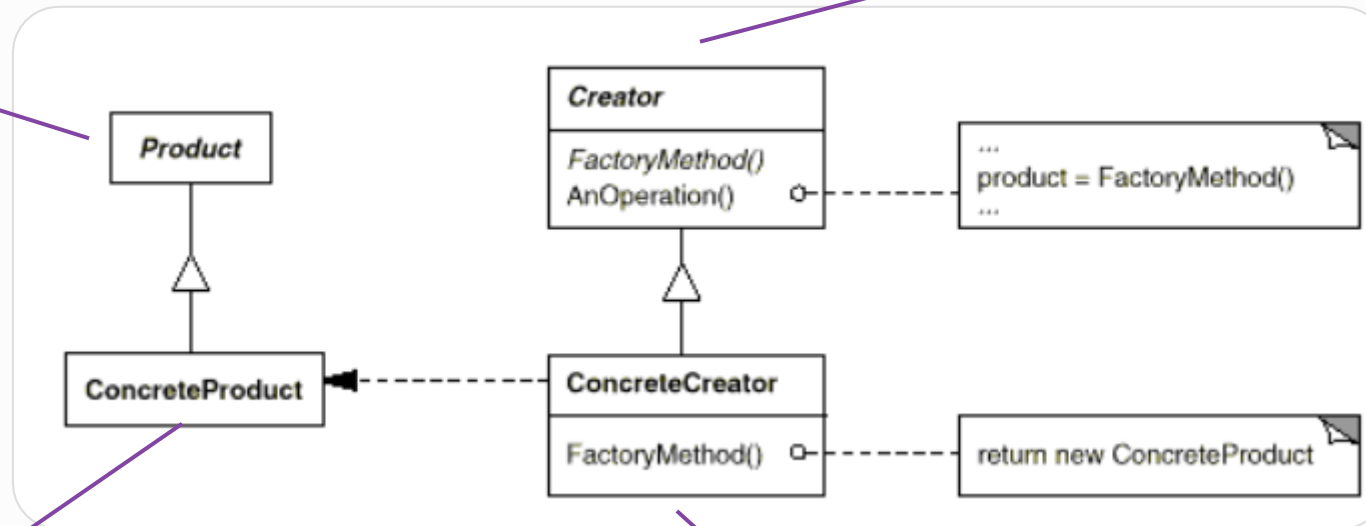
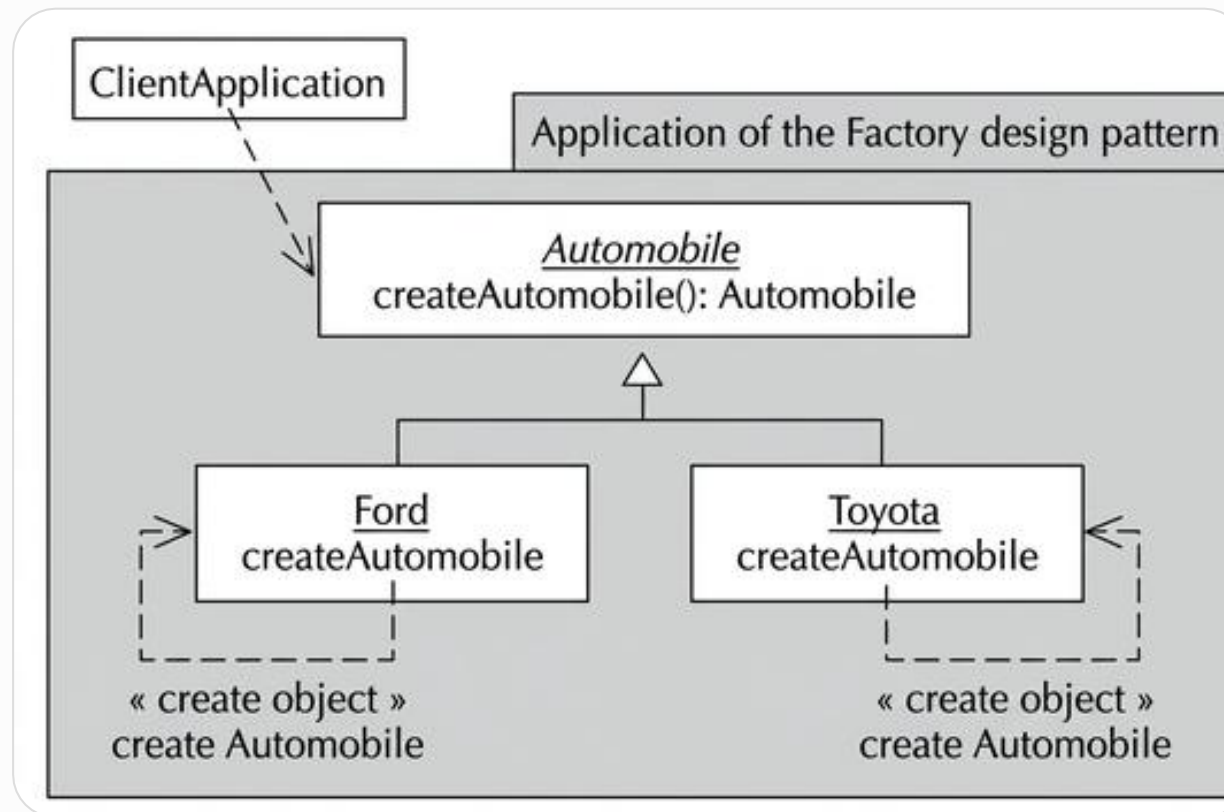


Image captured from the html version of the book published on CD (Gamma et al 1995)

- Implements the Product interface

- Overrides the factory method to return an instance of ConcreteProduct

# Code example of the Factory Method Design Pattern



Braude, Eric, 2005

# Code example of the Factory Method Design Pattern

```
abstract class Car // this is the Product class
{
    int buildDate;
    int getBuildDate()
    {
        return this.buildDate;
    }
    . . . // Other car attributes go here
    public double calculateInsurance(int buildDate)
    {
        return 0.00; // default class behavior
    }
}

class Ford extends Car // this is the ConcreteProduct class
{
    . . .
    double calculateInsurance()
    {
        return this.buildDate + this.buildDate*3.123/2.12; // Ford specific rules
    }
    . . .
}

class Toyota extends Car // this is another ConcreteProduct class
{
    . . .
    double calculateInsurance()
    {
        return this.buildDate*2 - this.buildDate*4.321/2.35; // Toyota specific rules
    }
    . . .
}
```

# Code example of the Factory Method Design Pattern

```
abstract class Brokerage // this is the Creator class
{
public Car createCar() // factory method
{
}
void renewInsurance() // anOperation
{
Car aut = createCar();
int year = aut.getBuildDate();
double v = aut.calculateInsurance();
System.out.println( "Insurance renewed!!" );
}
}

class CreatorFord extends Creator // this is the ConcreteCreator class
{
Ford criarCar()
{
return new Ford();
}
}

class CreatorToyota extends Creator // another ConcreteCreator class
{
Toyota createCar()
{
return new Toyota();
}
}
```

# Code example of the Factory Method Design Pattern

```
public static void main( String[] args )
{
    Car myCar = getCarType();
    Broker broker = new Broker();
    broker.renewInsurance(myCar); }

private static Car getCarType()
{
    System.out.println( "Please choose 1 for Ford or 2 for Toyota);
    try // selects user input
    { BufferedReader bufReader =
      new BufferedReader( new InputStreamReader( System.in ) );
      desiredType = bufReader.readLine();
    }
    catch( IOException e )
    { System.out.println( e );
    }
    // Returns the 'Car' instance according to user input
    // this is the section that needs to be changed if you want to make more car
    types available
    if( desiredType == 1)
    selectedCar = FordCreator.createCar();
    else
    selectedCar = ToyotaCreator.createCar();
    return selectedCar;
}
```

# Consequences of using the Factory Method

- Avoids strong coupling between the creator and concrete products
- It becomes easier to include new types of concrete products, without breaking the code
- **Downside:**
  - The code becomes more complex as it is necessary to include new classes to implement the pattern

## Patterns related to the Factory Method

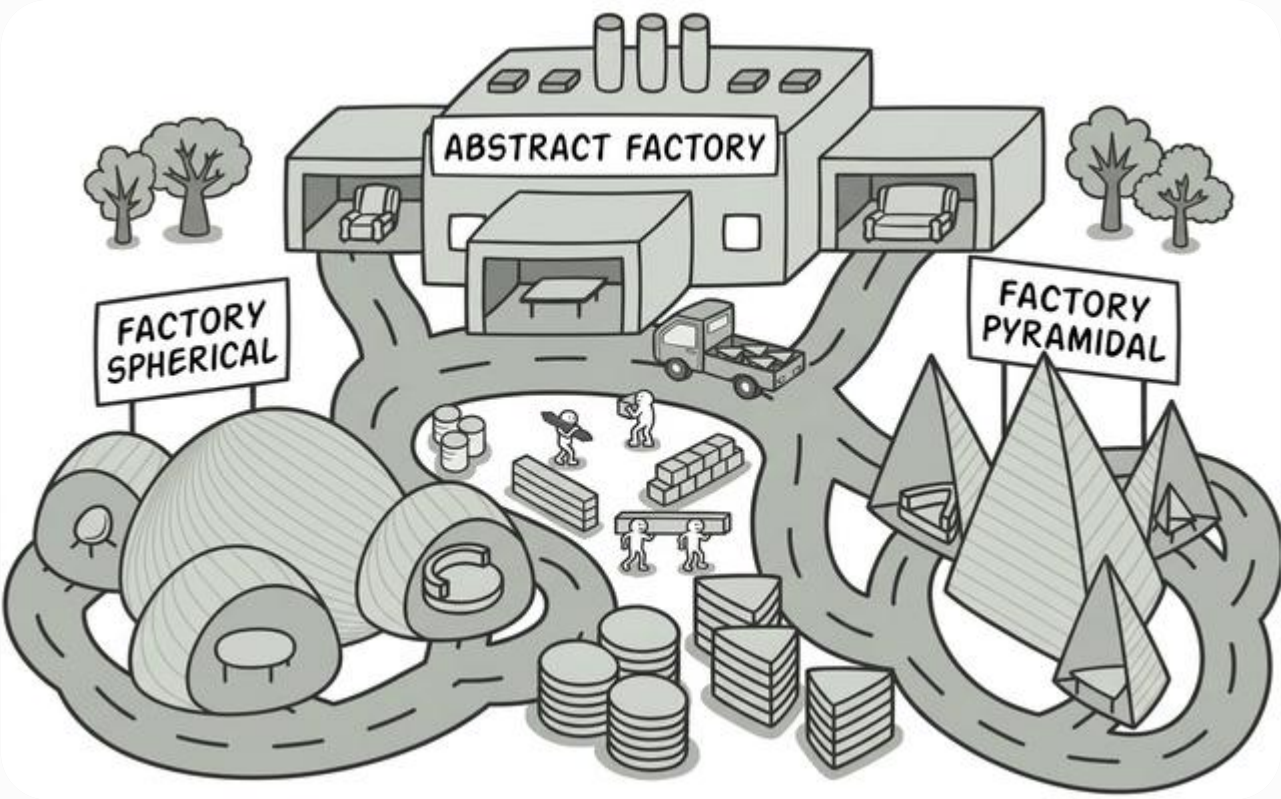
- Abstract Factory (we will see next!)
- Template Method
- Prototype



**Poll 4:** Use of the factory method



# Abstract Factory Design Pattern



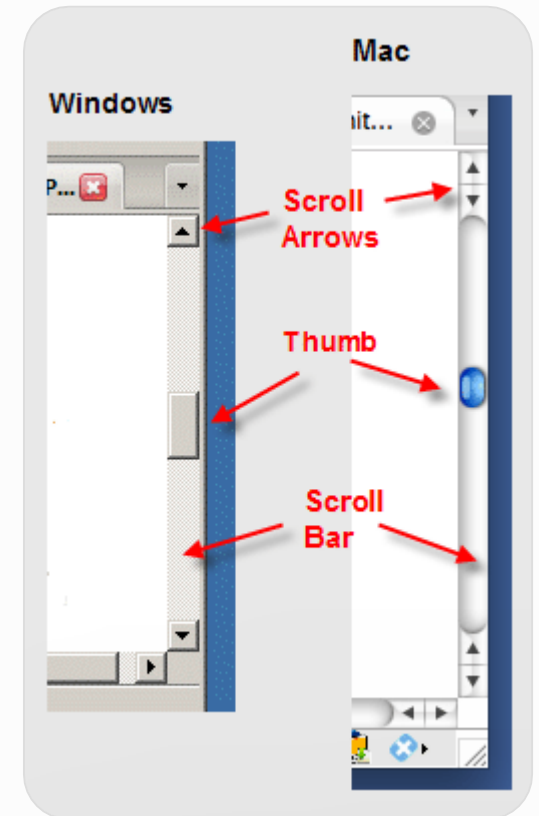
Shvets, Alexander, 2019

What if I need to create families of related objects?

|           | Chair | Sofa | Coffee table |
|-----------|-------|------|--------------|
| Art Deco  |       |      |              |
| Victorian |       |      |              |
| Modern    |       |      |              |

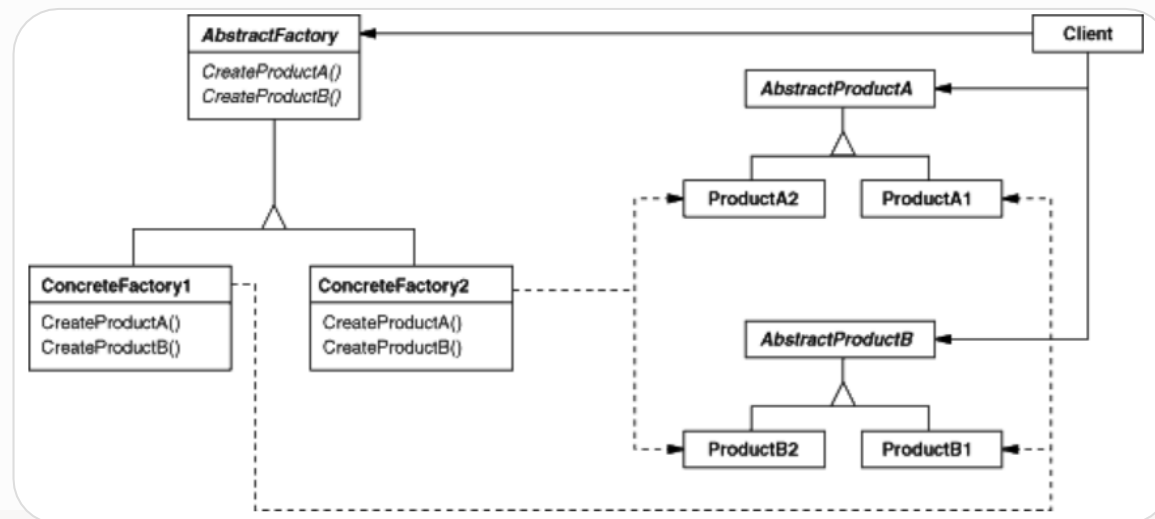
# Abstract Factory Design Pattern

- **Classification:** Structural Pattern/Object Scope
- **Intent:** Provide an interface for creating families of related or dependent objects without specifying their concrete classes.
- **Motivation:** In many applications, it is necessary to create a set of related objects, but which may have different implementations on each platform.
  - For example, in a graphical user interface system there may be “widgets” such as buttons, windows, scrollbars for Windows or for MacOS. If portability between different OSs is required, the code will be tied to certain classes and needs to be entirely modified
  - Once a style is chosen, all elements must always be used together
  - But there are common behaviors among elements of different families, for example: a button always has the same types of action: click (left and right), scrolling, etc., whether Windows or MacOS.



# Abstract Factory Design Pattern

- **Applicability** - Use the Abstract Factory pattern when:
  - a system should be independent of how its products are created, composed, and represented.
  - a system should be configured with one of several product families.
  - a family of related product objects is designed to be used together, and you need to enforce this constraint.
  - you want to provide a library of product classes, and you want to reveal only their interfaces, not their implementations.
- **Structure:**



*Image captured from the html version of the book published on CD (Gamma et al 1995)*

# Abstract Factory Design Pattern

- Participants:

- Declares an interface for operations that create abstract objects

- Implements the operations to create the concrete objects

- Declares an interface for a product type

Uses only interfaces declared by AbstractFactory and AbstractProduct

- Defines an object to be created by the corresponding concrete factory
- Implements the AbstractProduct interface

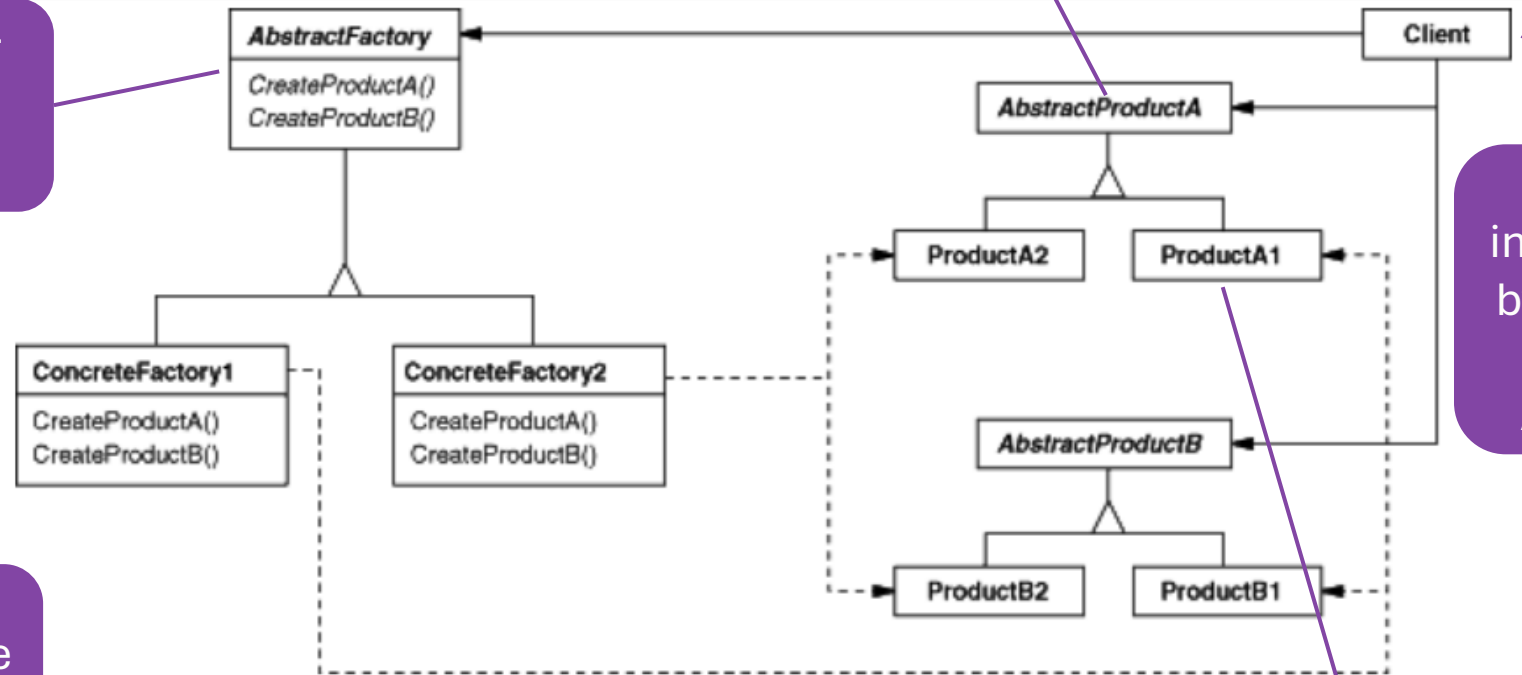


Image captured from the html version of the book published on CD (Gamma et al 1995)

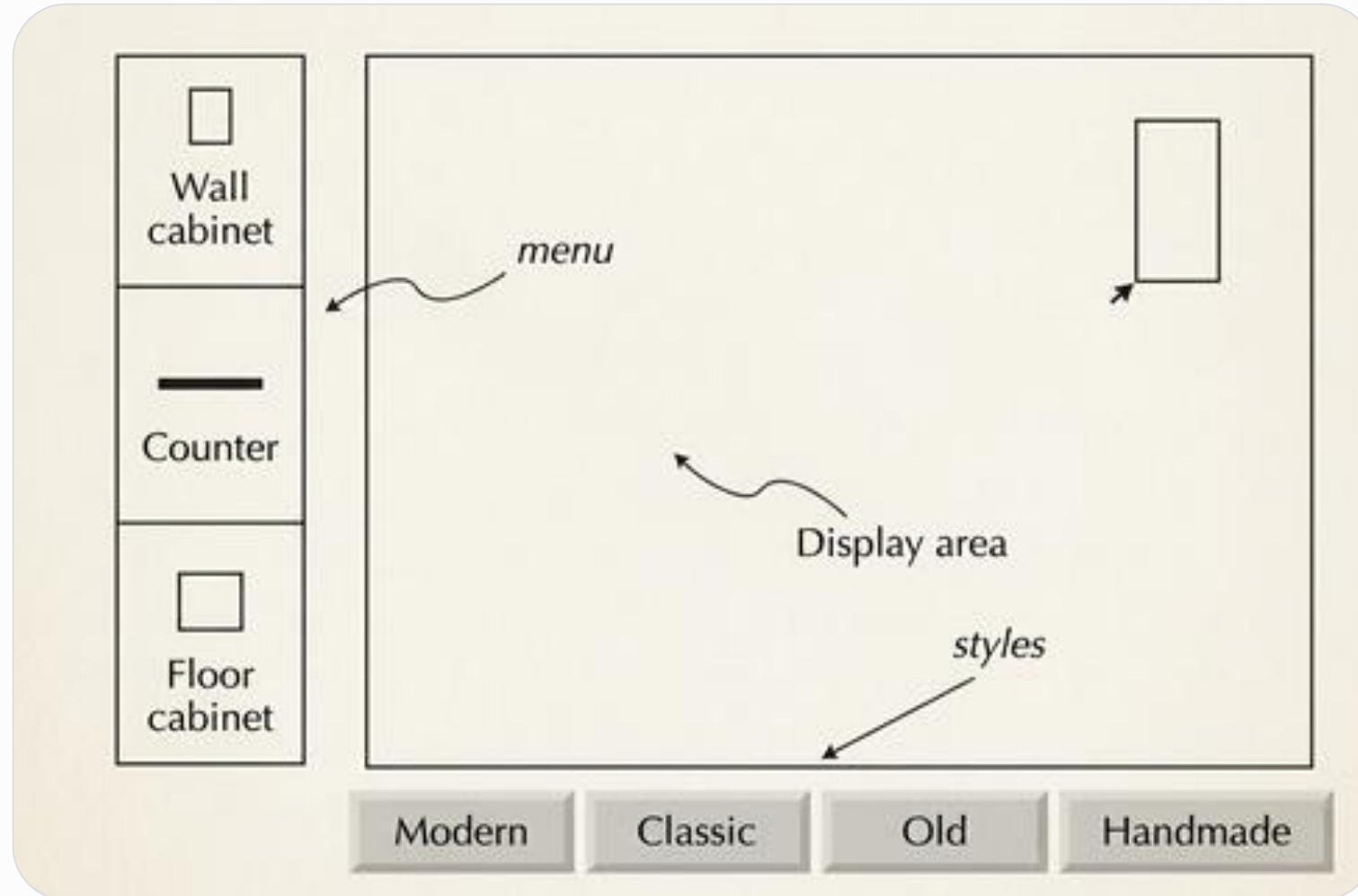
# Abstract Factory Design Pattern

## Abstract Factory Pattern Example

- Application for building kitchen cabinets
- Homeowners always intend to modernize their kitchens, often using software to visualize the possibilities.
- Kitchen Visualizer: an application that allows the user to create the layout of kitchen parts, without committing to a style.

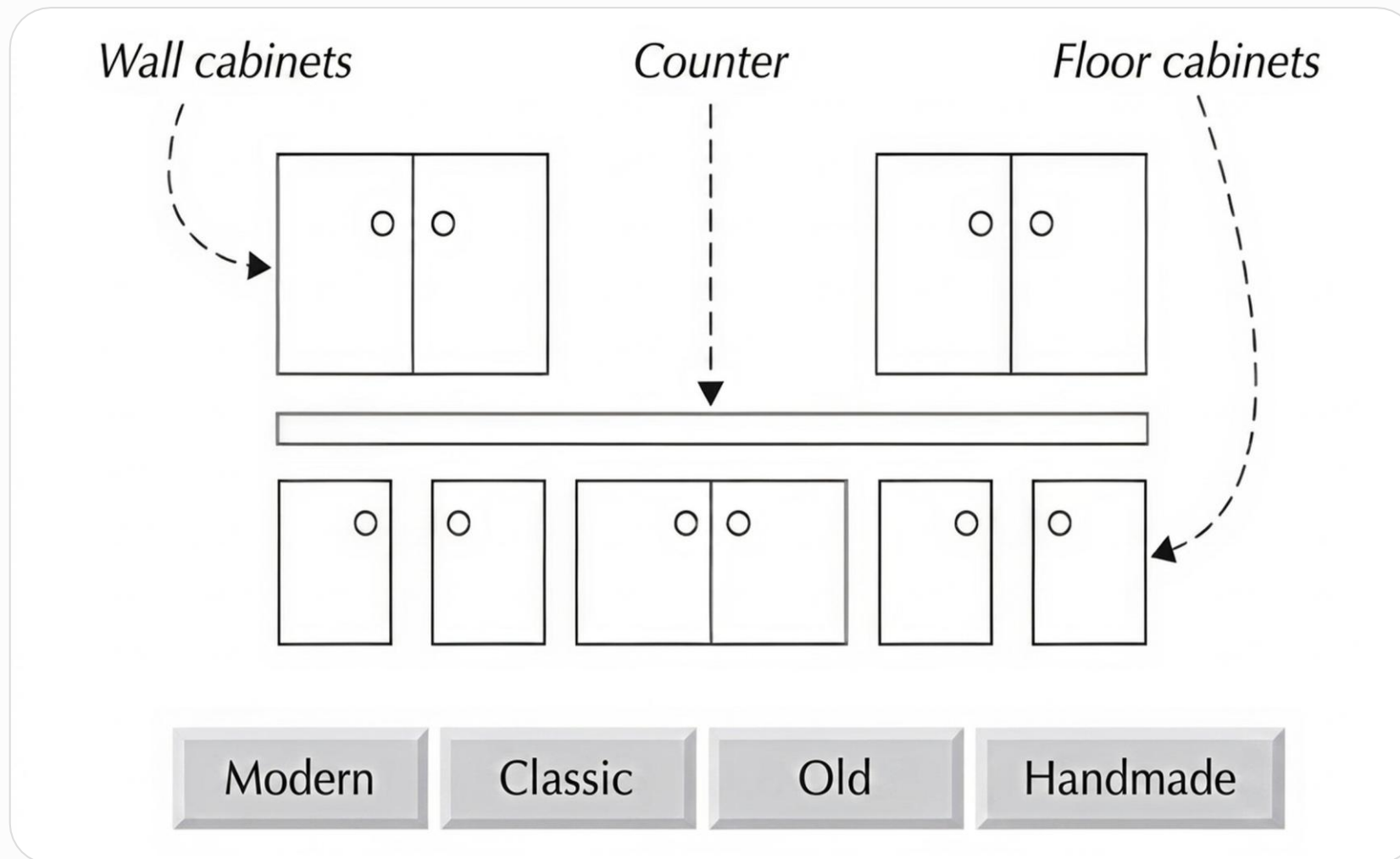
# Abstract Factory Design Pattern

## Abstract Factory Pattern Example



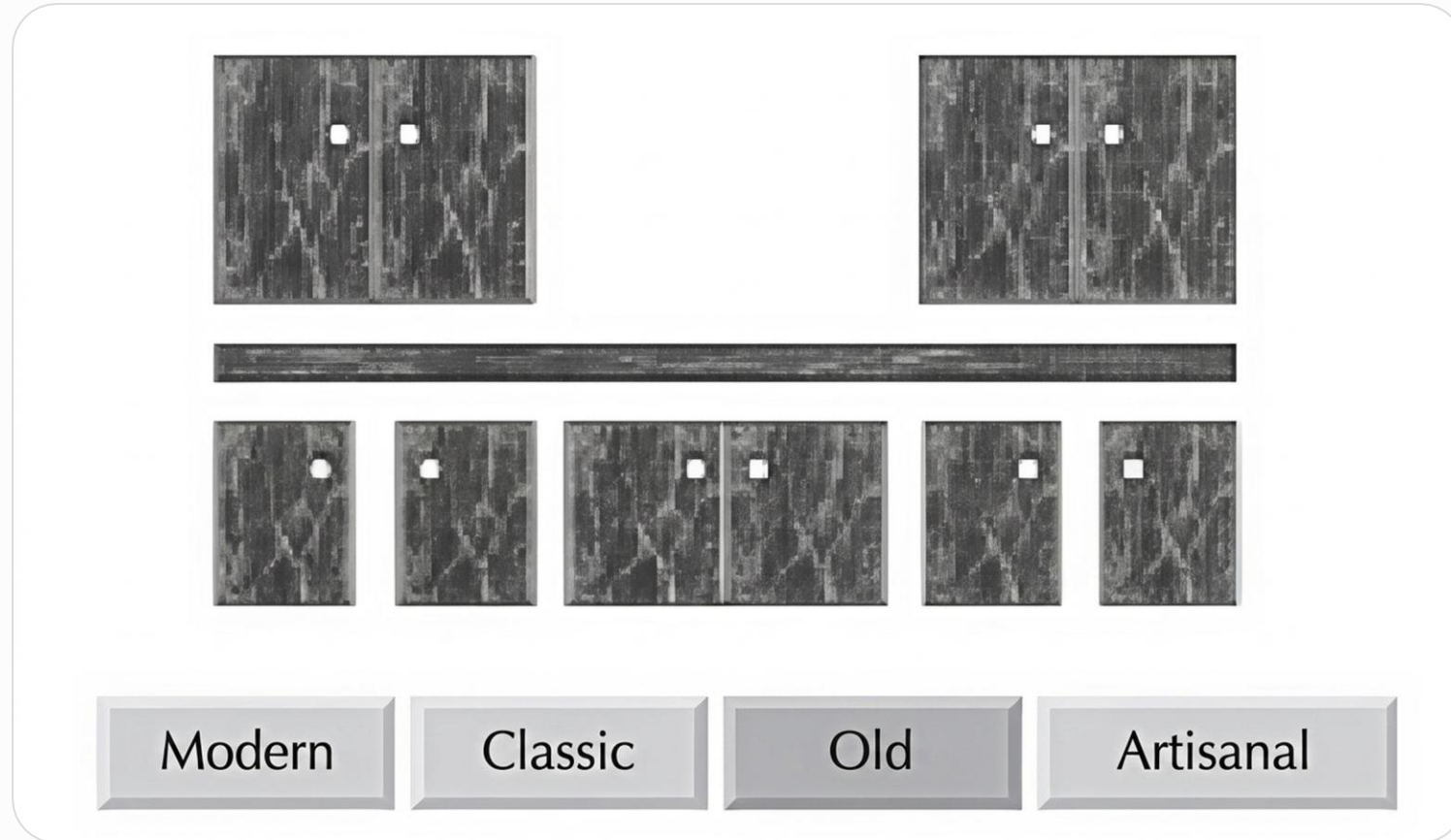
# Abstract Factory Design Pattern

## Abstract Factory Pattern Example



# Abstract Factory Design Pattern

## Abstract Factory Pattern Example

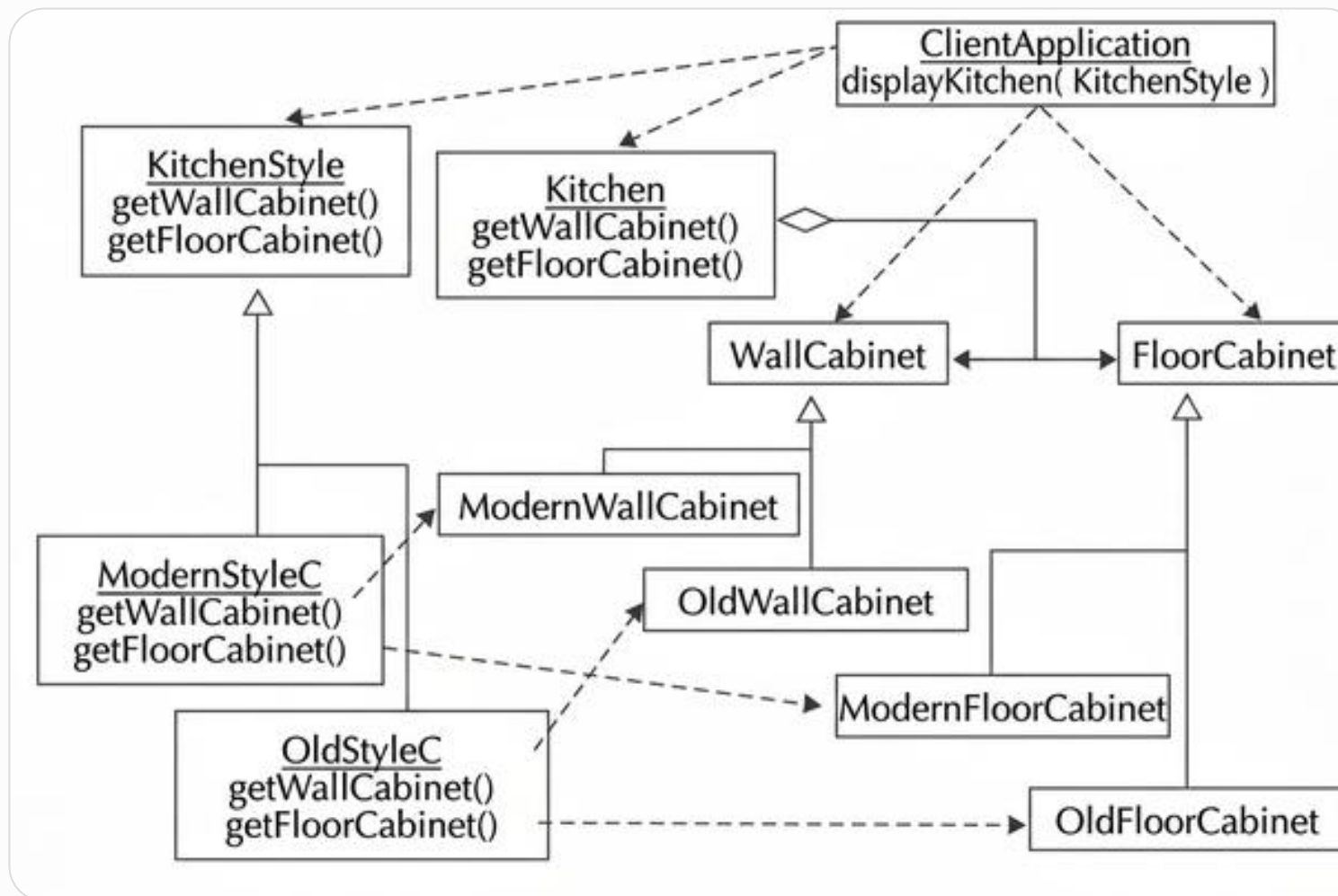


# Abstract Factory Pattern Example



# Abstract Factory Design Pattern

## Abstract Factory Pattern Example



# Abstract Factory Design Pattern

## Abstract Factory Pattern Example

```
//Determines the style by instantiating myStyle
OldStyleK myStyle = new OldStyleK;
// Creates wall cabinets: Type determined by myStyle's class
WallCabinet WallCabinet1 = myStyle.getWallCabinet();
WallCabinet WallCabinet2 = myStyle.getWallCabinet();
. . .
// Creates base cabinets: Type determined by myStyle's class
// Creates the kitchen object (in the required style)
BaseCabinet BaseCabinet1 = myStyle.getBaseCabinet();
BaseCabinet BaseCabinet2 = myStyle.getBaseCabinet();
. . .
Kitchen kitchen = new Kitchen();
Kitchen.add( WallCabinet1, . . . );
Kitchen.add( WallCabinet2, . . . );
. . .
Kitchen.add( BaseCabinet1 . . . );
Kitchen.add( BaseCabinet2 . . . );
. . .
```

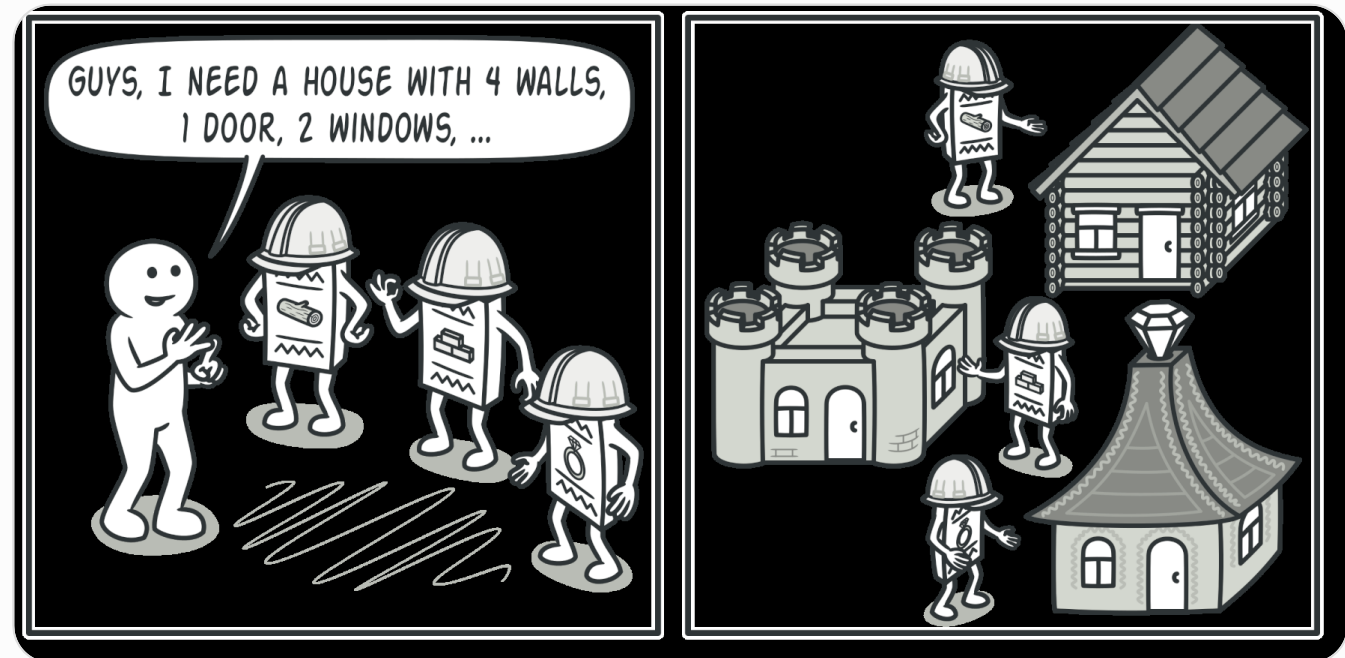
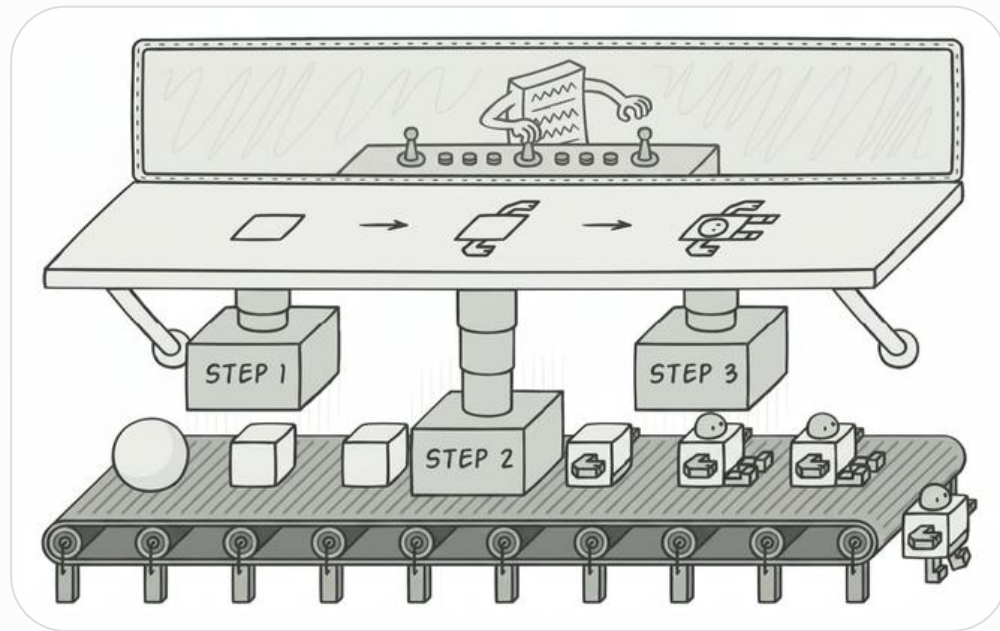
# Abstract Factory Design Pattern

## Abstract Factory Pattern Example

```
//Determines the style by instantiating myStyle
ModernStyleK myStyle = new ModernStyleK;
// Creates wall cabinets: Type determined by myStyle's class
WallCabinet WallCabinet1 = myStyle.getWallCabinet();
WallCabinet WallCabinet2 = myStyle.getWallCabinet();
. . .
// Creates base cabinets: Type determined by myStyle's class
// Creates the kitchen object (in the required style)
BaseCabinet baseCabinet1 = myStyle.getBaseCabinet();
BaseCabinet baseCabinet2 = myStyle.getBaseCabinet();
. . .
Kitchen kitchen = new Kitchen();
Kitchen.add( wallCabinet1, . . . );
Kitchen.add( wallCabinet2, . . . );
. . .
Kitchen.add( baseCabinet1 . . . );
Kitchen.add( baseCabinet2 . . . );
. . .
```

# Builder Design Pattern

- **Classification:** Creational Pattern/Object Scope
- **Intent:** Separate the construction of a complex object from its representation so that the same construction process can create different representations.



(Shvets, Alexander, 2018)

# Builder Design Pattern

- Structure:

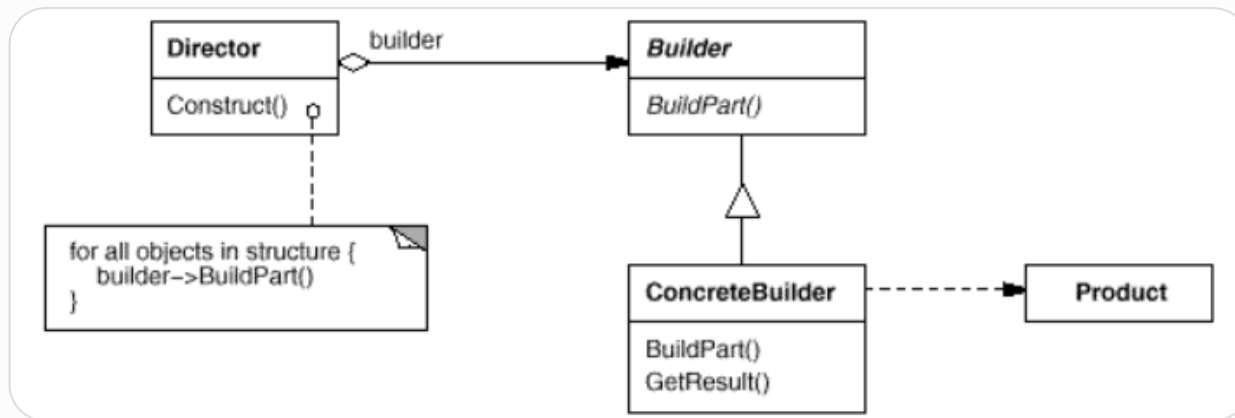
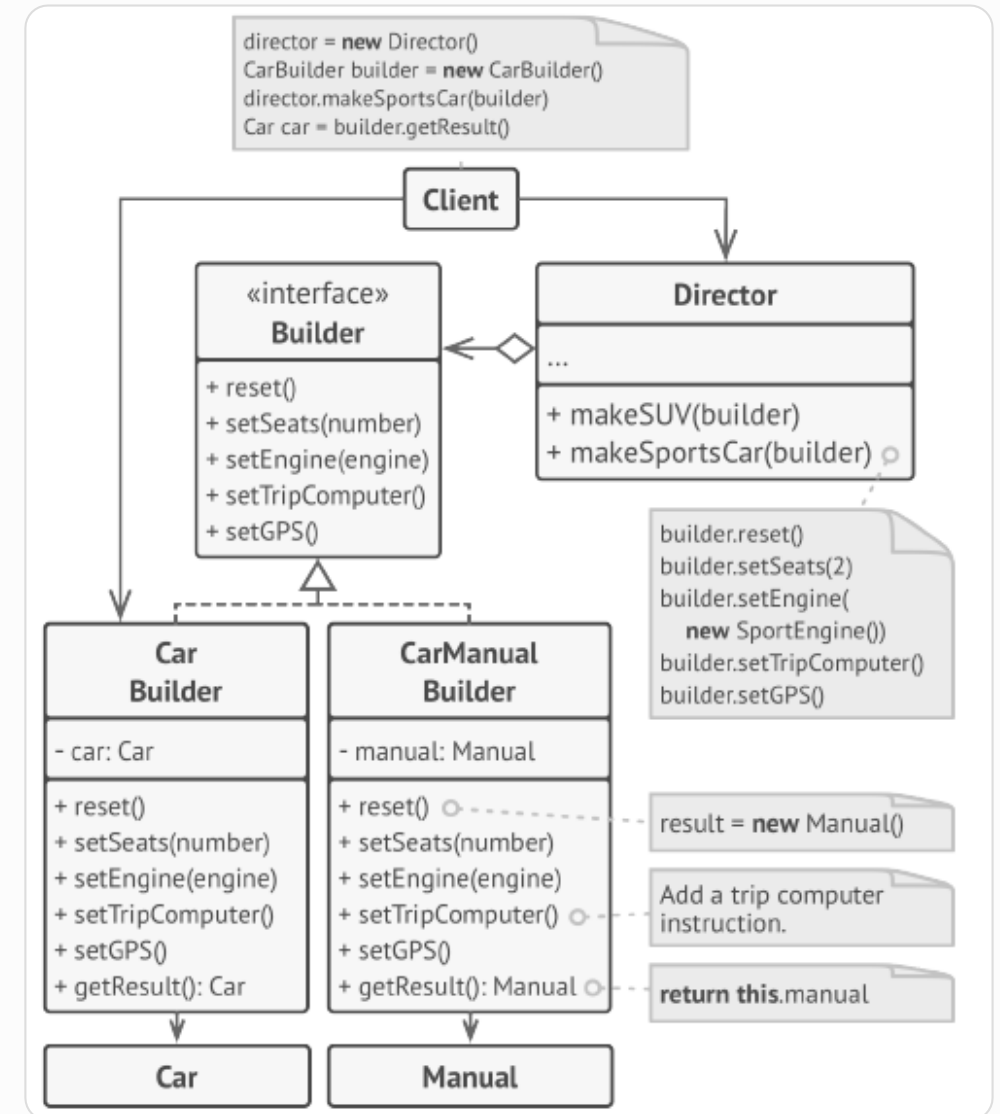


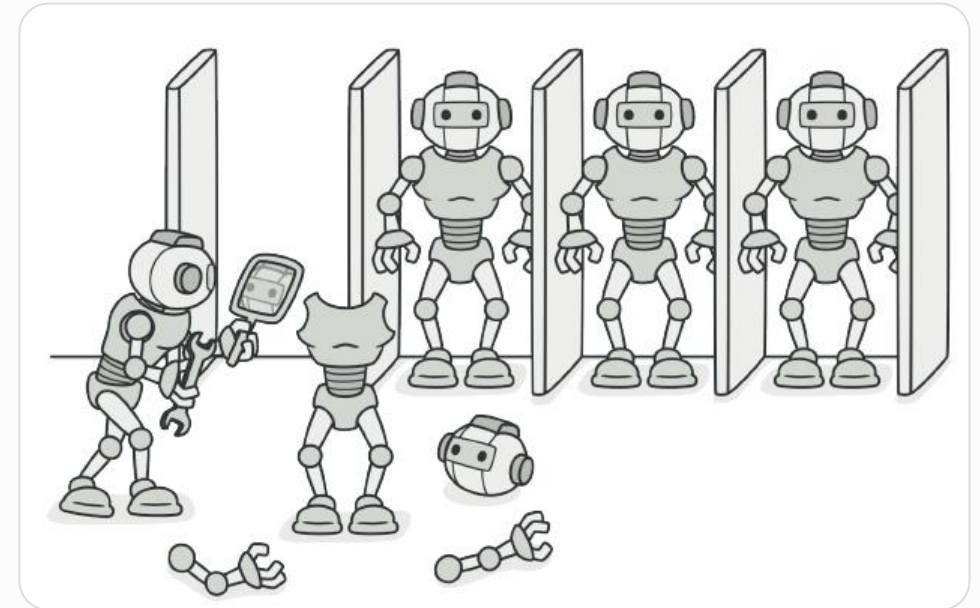
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

# Prototype Design Pattern

- **Classification:** Creational Pattern/Object Scope
- **Intent:** Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype, making the code independent of the class of these objects
- **Applicability:** cases where objects are passed through third-party code via some interface. The concrete classes of these objects are unknown, and you couldn't depend on them even if you wanted to.



(Shvets, Alexander, 2018)

# Prototype Design Pattern

- Structure:

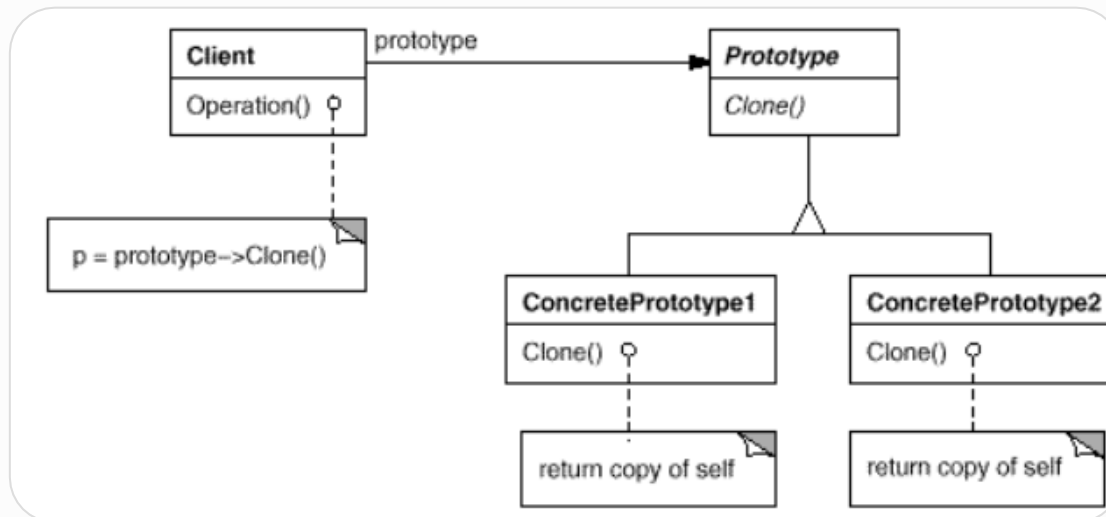
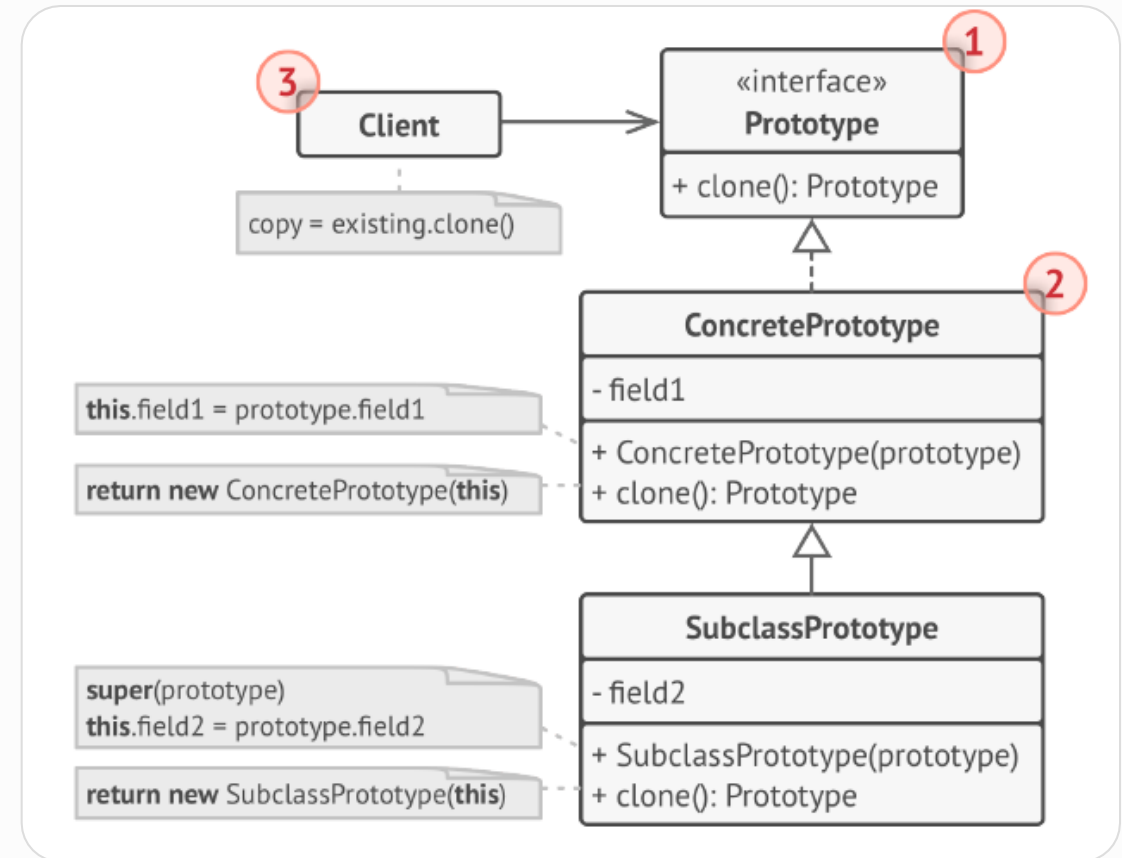


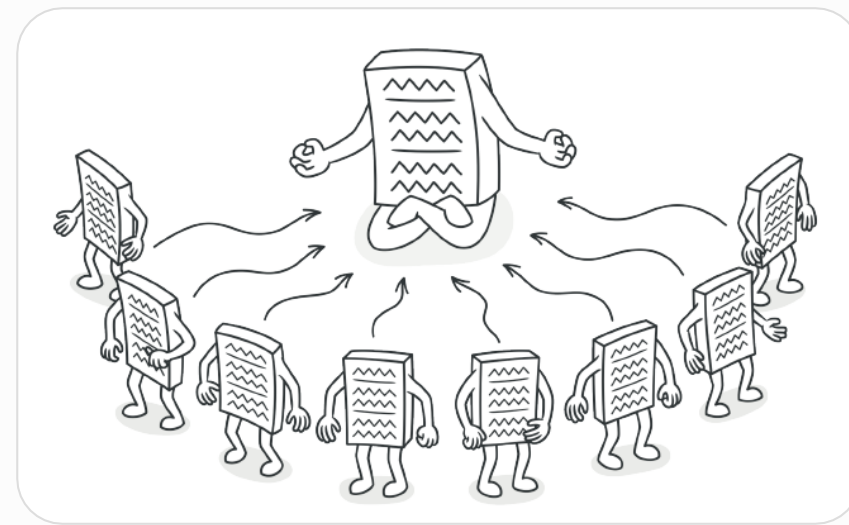
Image captured from the html version of the book published on CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

# Singleton Design Pattern

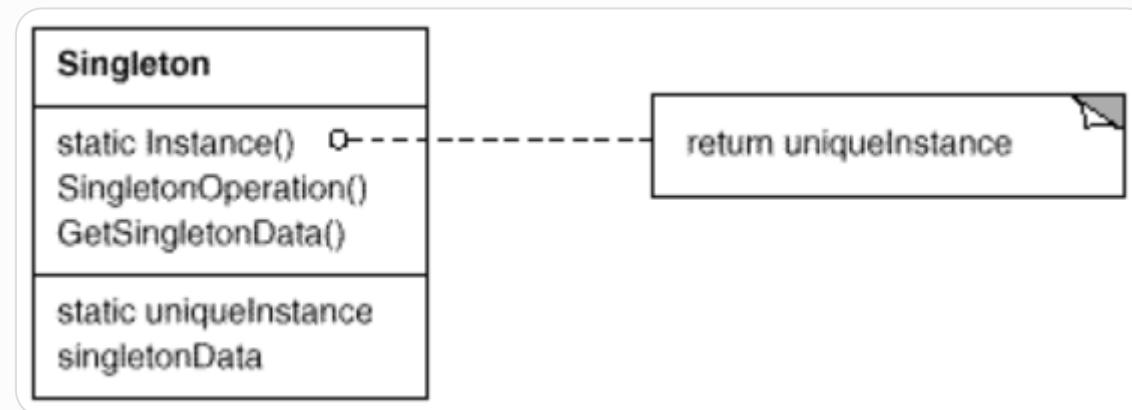
- **Classification:** Creational Pattern/Object Scope
- **Intent:** Ensure a class has only one instance and provide a global point of access to it.
- **Motivation:** In some applications, it is important to ensure that some classes have exactly one instance, for example, only one file system and one window manager.
  - Could a global variable solve the problem?



(Shvets, Alexander, 2018)

# Singleton Design Pattern

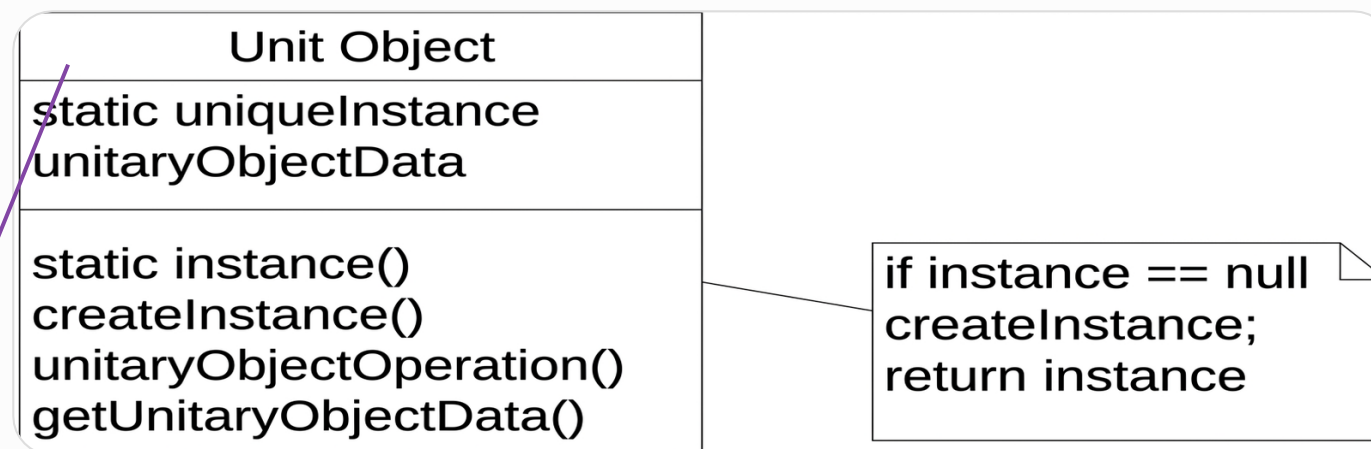
- **Applicability**
- Use the Singleton pattern when:
  - there must be exactly one instance of a class, and it must be easily accessible to clients.
  - the sole instance must be extensible by subclassing, and clients should be able to use an extended instance without modifying their code.
- **Structure:**



*Images captured from the HTML version of the book published on CD (Gamma et al 1995)*

# Singleton Design Pattern

- Participant:



(Braude, Eric, 2005)

- Defines an Instance() operation (in the class) that lets clients access its unique instance.

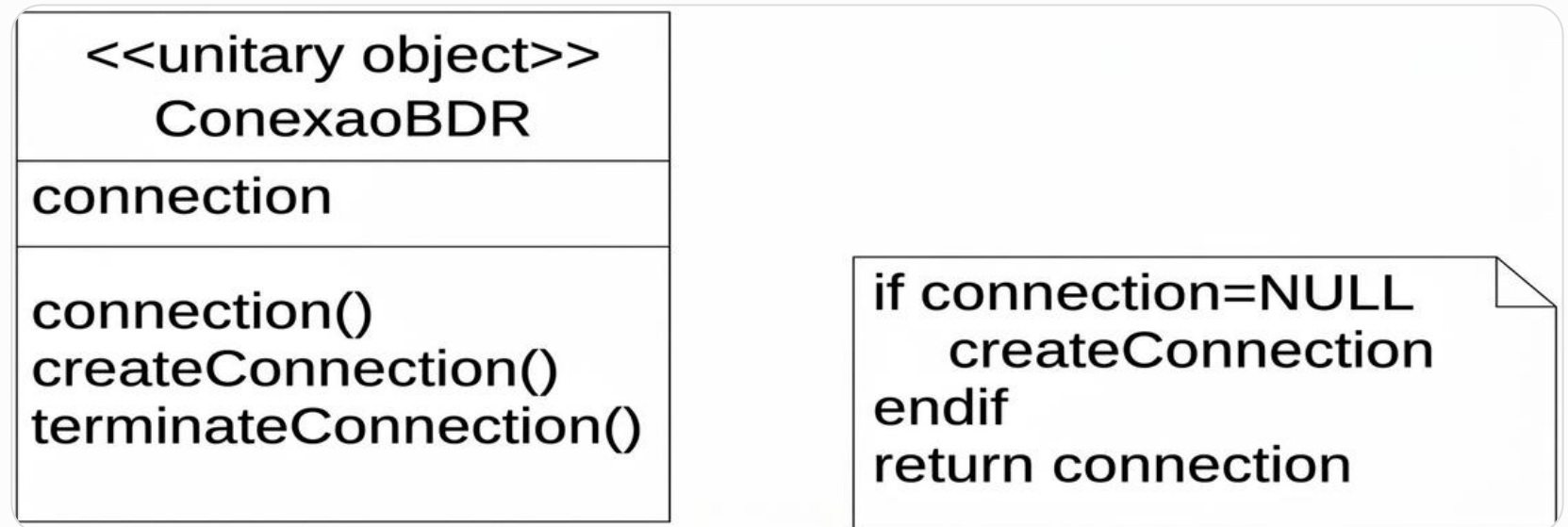
# Singleton Design Pattern

## Singleton Pattern Example

- When starting an application, it is necessary to establish a connection with the database, to have an open communication channel during the application's execution.
- Generally, this is implemented through a Connection class (several existing DBMS interfaces provide a Connection class specifically for this purpose).
- This connection must subsequently be closed when the application terminates.
- The connection to the DBMS should preferably be unique, because if a new connection is created every time an object is created, numerous objects will exist in memory unnecessarily.
- Thus, the Singleton pattern can be applied to ensure that for each application running, a single valid connection to the DBMS exists.

# Singleton Design Pattern

## Singleton Pattern Example



# Singleton Pattern Example

```
// final -> prevents inheritance
public final class ConnectionPool {

    private static final String DATA_SOURCE_MYSQL = "java:comp/env/jdbc/passeLivre";
    private DataSource dataSource;           // database connection pool
    private static ConnectionPool myself;    // reference to a single instance of this class

    // private constructor
    private ConnectionPool( DataSource dataSource ) {

        this.dataSource = dataSource;
    }
    // synchronized to prevent more than one instance from being created in a multithreaded sys
    public static synchronized ConnectionPool getInstance() {

        try {
            // checks if a single instance has not yet been created
            if( myself == null ) {

                // gets the application context
                Context contexto = new InitialContext();
                // gets the database connection pool
                DataSource dataSource = ( DataSource )contexto.lookup( DATA_SOURCE_MYSQL );
                // creates the single instance of this class
                myself = new ConnectionPool( dataSource );
            }
        } catch( NamingException e ) {

            System.err.println( e.getMessage() );
        }

        return myself;
    }

    public Connection getConnection() throws SQLException {

        return dataSource.getConnection();
    }
}
```



Poll 5 : Singleton Usage



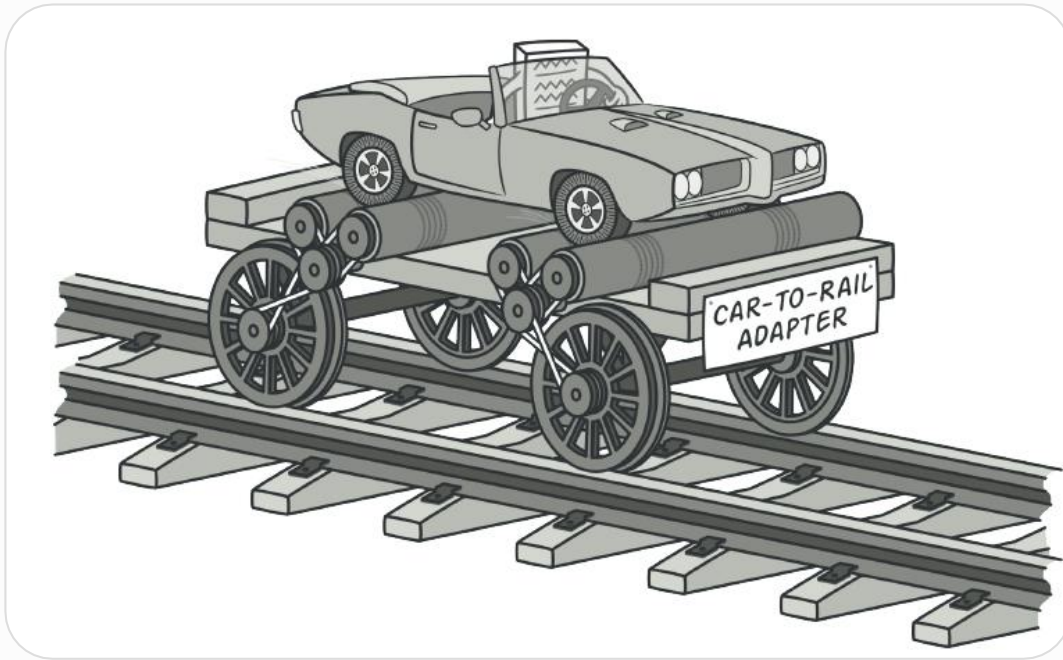
|       |        | Purpose                                                                                                                           |                                                                                                                                                                                 |                                                                                                                                                                                                                                       |
|-------|--------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |        | Creational                                                                                                                        | Structural                                                                                                                                                                      | Behavioral                                                                                                                                                                                                                            |
| Scope | Class  | Factory Method                                                                                                                    | Adapter                                                                                                                                                                         | Interpreter<br>Template Method                                                                                                                                                                                                        |
|       | Object | <ul style="list-style-type: none"> <li>• Abstract Factory</li> <li>• Builder</li> <li>• Prototype</li> <li>• Singleton</li> </ul> | <ul style="list-style-type: none"> <li>• Adapter</li> <li>• Bridge</li> <li>• Composite</li> <li>• Decorator</li> <li>• Facade</li> <li>• Flyweight</li> <li>• Proxy</li> </ul> | <ul style="list-style-type: none"> <li>• Chain of Responsibility</li> <li>• Command</li> <li>• Iterator</li> <li>• Mediator</li> <li>• Memento</li> <li>• Observer</li> <li>• State</li> <li>• Strategy</li> <li>• Visitor</li> </ul> |

# PATTERNS of the “structural” category

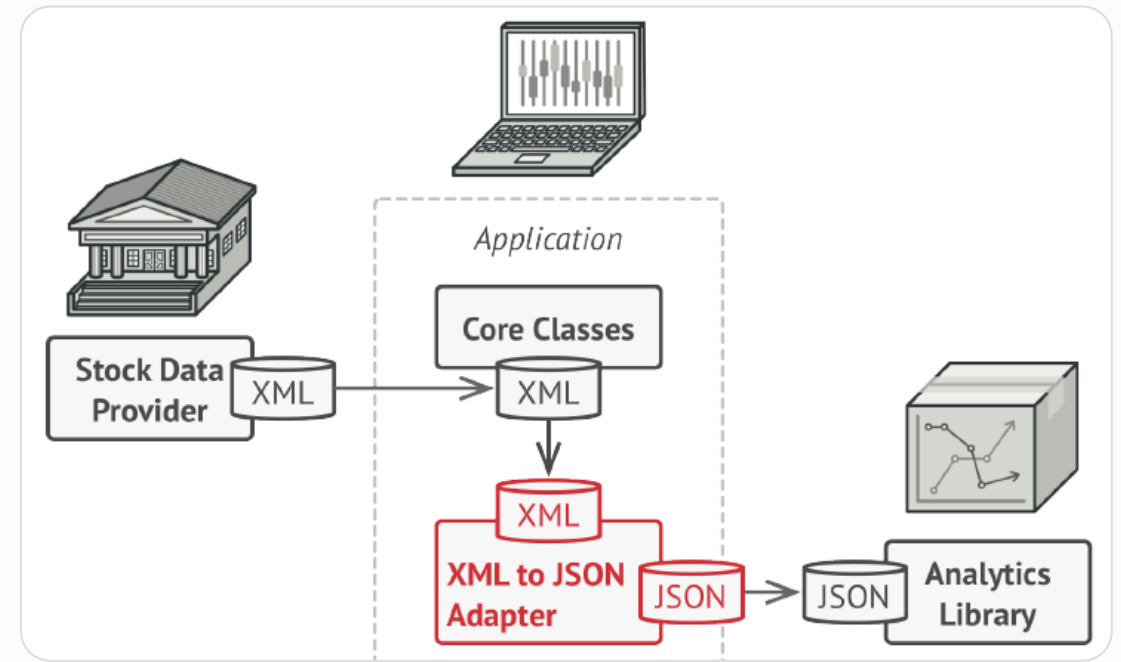
They are concerned with how compositions of classes and objects occur in larger structures

# Adapter Design Pattern

- **Classification:** Structural pattern/Class/object scope
- **Intent:** Convert the interface of a class into another interface that clients expect. The Adapter allows classes with incompatible interfaces to work together

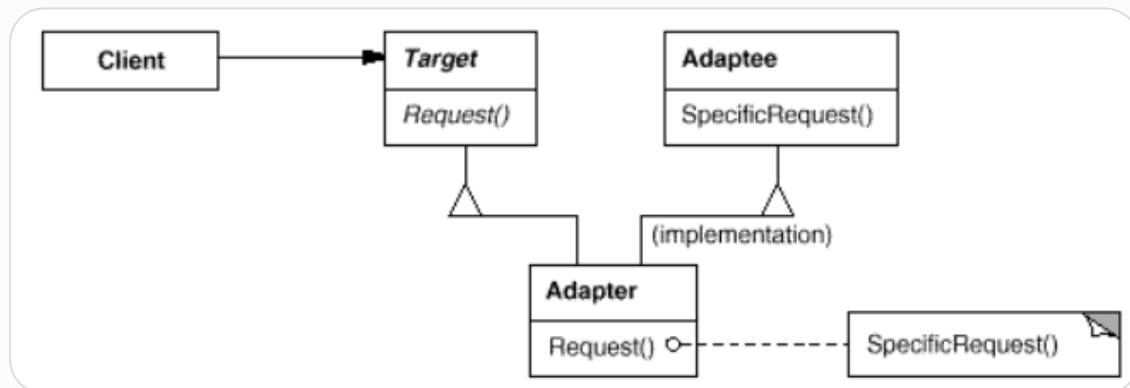


(Shvets, Alexander, 2018)

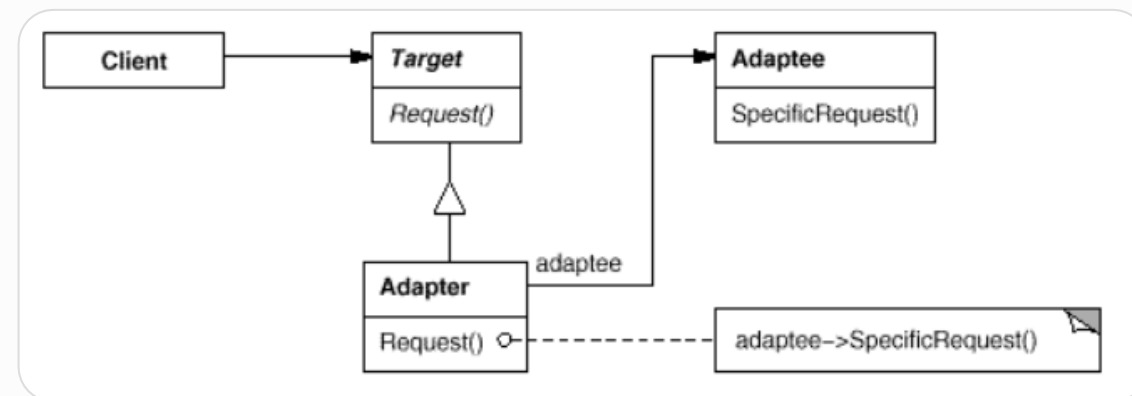


# Adapter Design Pattern

- Structure:



Class-scope Adapter

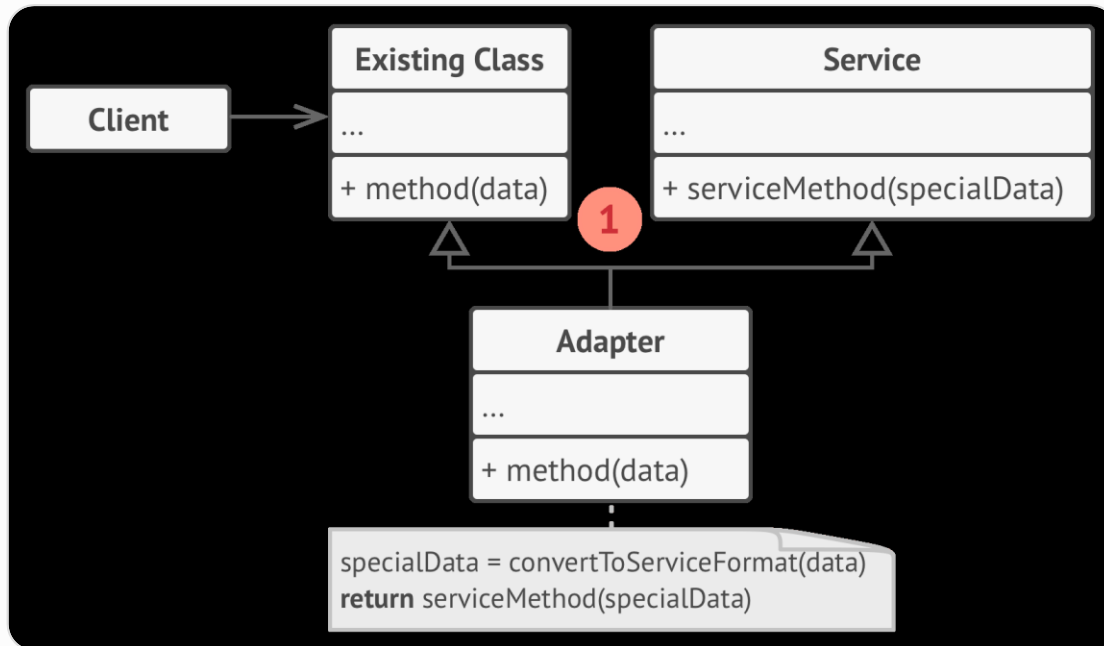


Object-scope Adapter

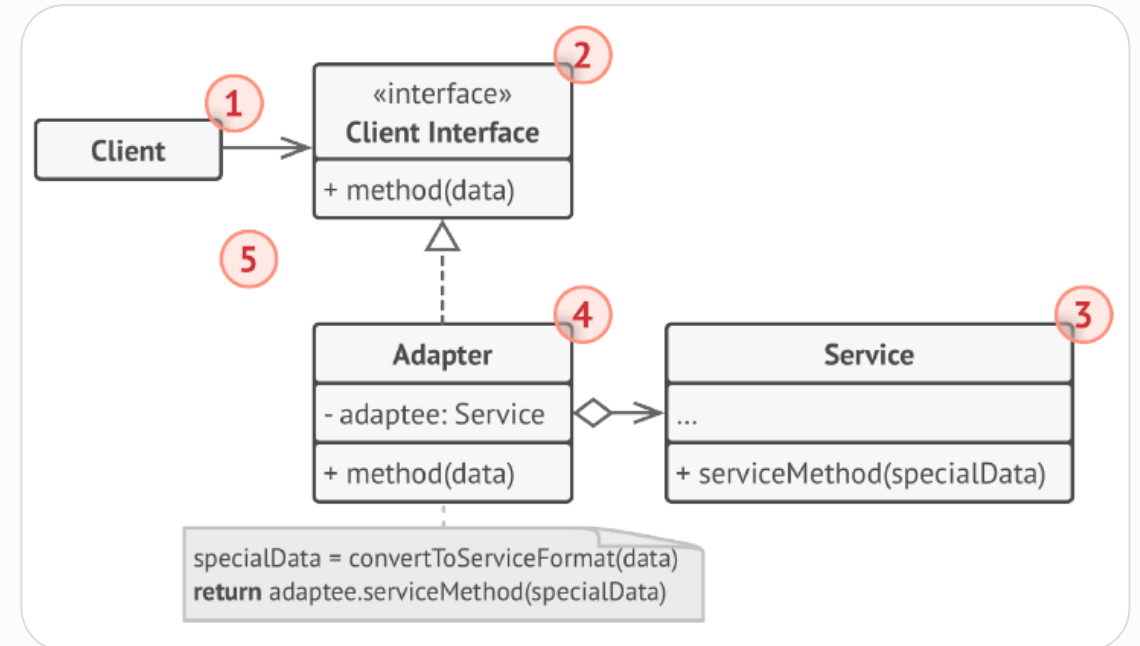
*Images captured from the HTML version of the book published on CD (Gamma et al 1995)*

# Adapter Design Pattern

- Structure:



Class-scope Adapter

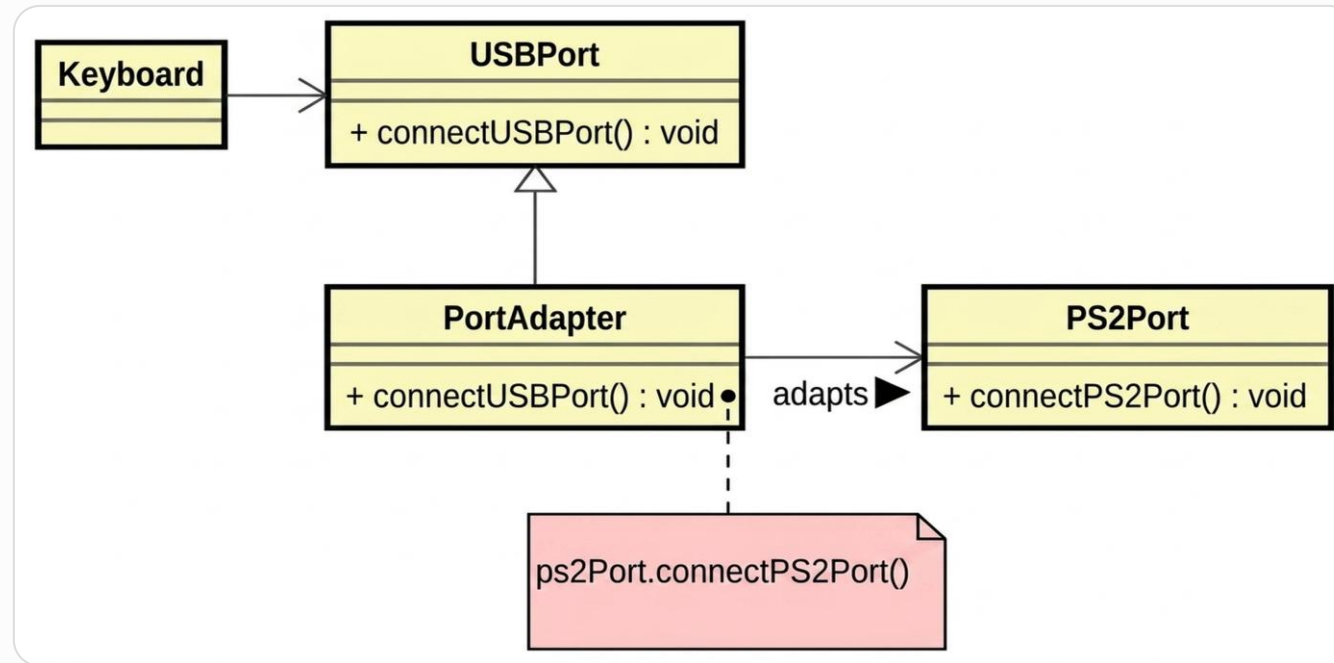


Object-scope Adapter

(Shvets, Alexander, 2018)

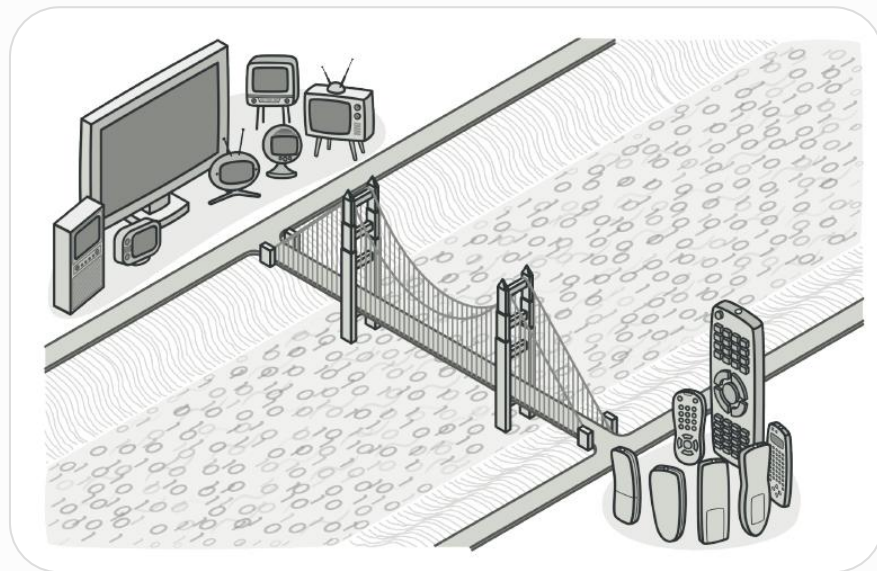
# Adapter Design Pattern

- Example:

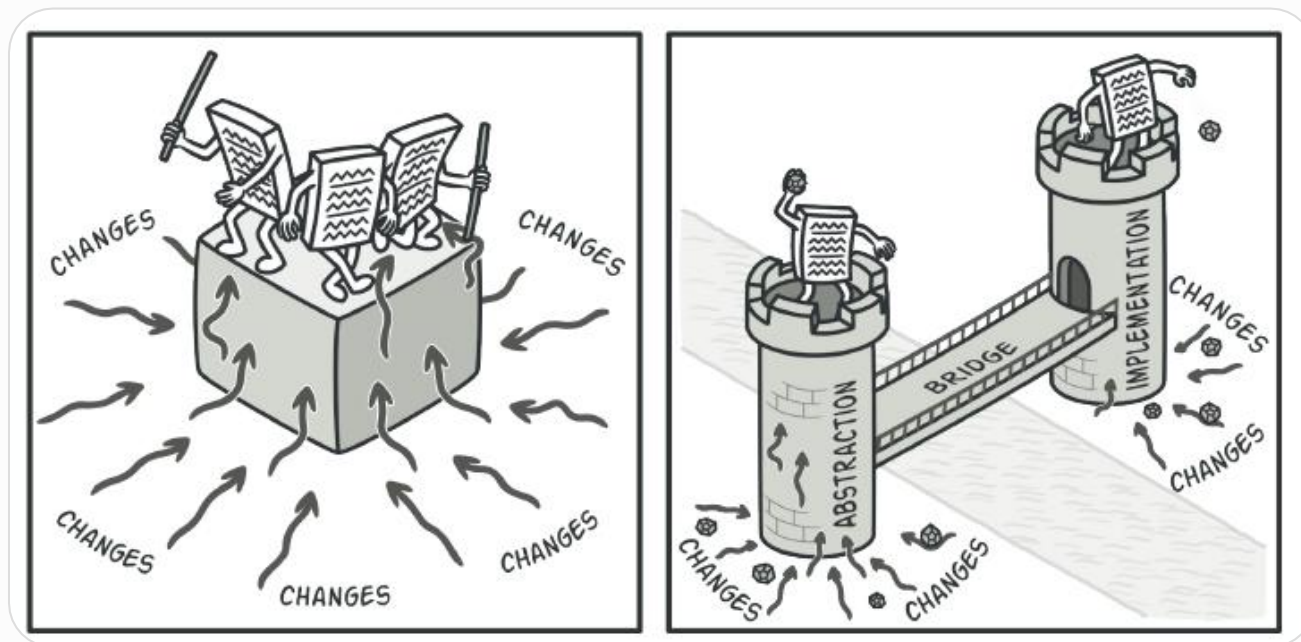


# Bridge Design Pattern

- **Classification:** Structural pattern/Object scope
- **Intent:** Decouple an abstraction from its implementation so that the two can vary independently.



(Shvets, Alexander, 2018)



# Bridge Design Pattern

- Structure:

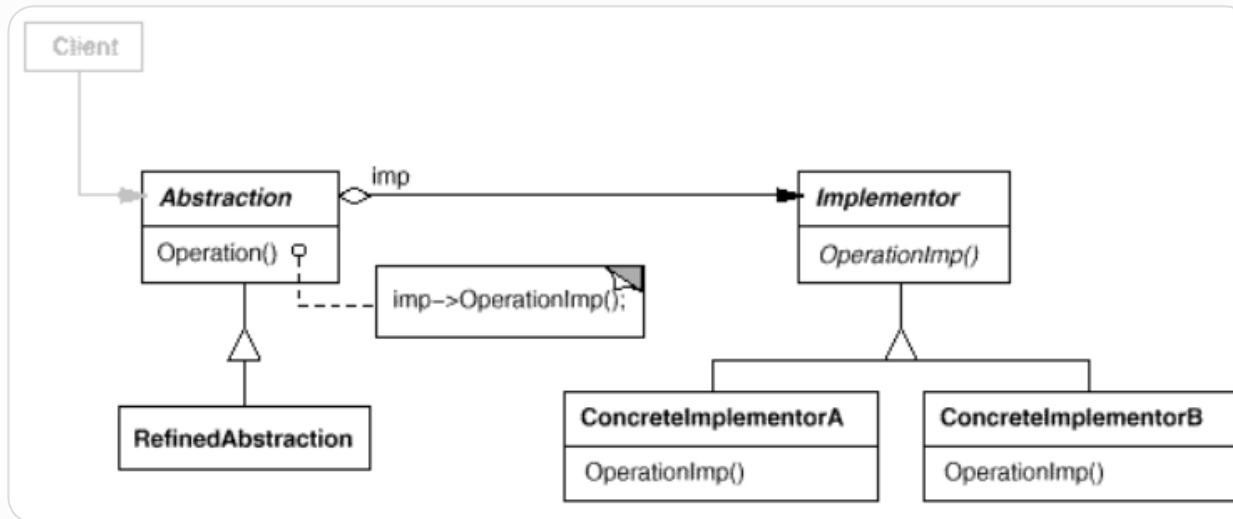
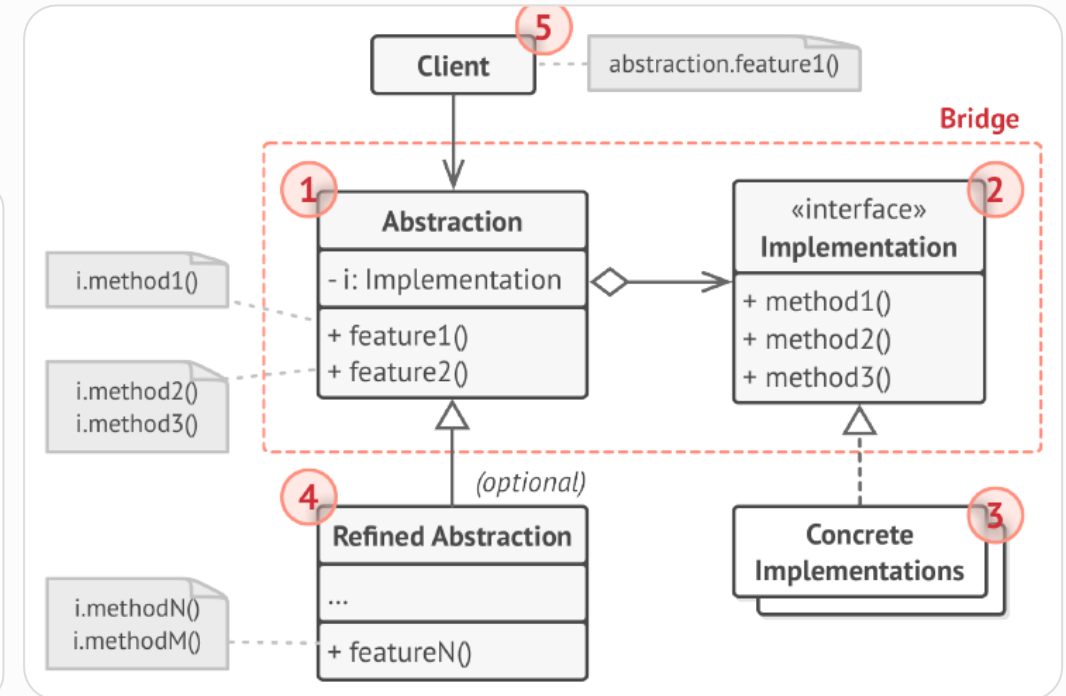


Image captured from the html version of the book published on CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

# Next class

- Structural and behavioral patterns

|       |        | Purpose                                                                                                                      |                                                                                                                                                                         |                                                                                                                                                                                                                             |
|-------|--------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |        | Creational                                                                                                                   | Structural                                                                                                                                                              | Behavioral                                                                                                                                                                                                                  |
| Scope | Class  | Factory Method                                                                                                               | Adapter                                                                                                                                                                 | Interpreter<br>Template Method                                                                                                                                                                                              |
|       | Object | <ul style="list-style-type: none"><li>• Abstract Factory</li><li>• Builder</li><li>• Prototype</li><li>• Singleton</li></ul> | <ul style="list-style-type: none"><li>• Adapter</li><li>• Bridge</li><li>• Composite</li><li>• Decorator</li><li>• Facade</li><li>• Flyweight</li><li>• Proxy</li></ul> | <ul style="list-style-type: none"><li>• Chain of Responsibility</li><li>• Command</li><li>• Iterator</li><li>• Mediator</li><li>• Memento</li><li>• Observer</li><li>• State</li><li>• Strategy</li><li>• Visitor</li></ul> |

# References

- Biggerstaff, Ted J. ; Perlis, Alan J. **Software Reusability: Concepts and Models**. Acm Press Frontier Series, 1989
- Alexander et al. **A Pattern Language**. New York: Oxford University Press, 1977.
- Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. **Design Patterns - Elements of Reusable Object-Oriented Software**. Addison-Wesley, 1995
- Meszaros, G.; Doble, J. **A pattern language for pattern writing**, cap. 29 in J. Coplien; D. Schmidt. Pattern Languages of Program Design, Reading-MA, Addison-Wesley, p. 529–574, 1998.
- Buschmann F. et. al. **Pattern-oriented software architecture - a system of patterns**. Wiley, 1996.
- Coplien, J. **Advanced C programming styles and idioms**. Reading-MA: Addison-Wesley, 1992.
- Shvets, Alexander. **Dive into design patterns**, Refactoring.Guru, 2018, available for download at: <https://www.goodreads.com/book/show/43125355-dive-into-design-patterns>
- Braude, Eric. **Projeto de Software: Da programação à arquitetura: Uma abordagem baseada em Java**, Bookman, 2005.

# Recommended readings

- The website <https://refactoring.guru/design-patterns> has interesting material on patterns, including examples implemented in different programming languages. One of the books I used in preparing the slides is available online:
  - Dive into design patterns - <https://refactoring.guru/design-patterns/catalog>
- The book Gamma et al, 1995 was used as a basis for preparing the classes. We have a copy in the ICMC-USP and IME-USP libraries.
- The book Braude, 2005 was also used, with fewer references.
- The website <https://hillside.net> contains excellent material on software patterns in general, particularly Design Patterns. It contains articles published in all pattern conferences held in the United States and in various places around the world, including Brazil (SugarLoafPLoP). [The Hillside Group - A group dedicated to design patterns. Home of the patterns library.](#)

# Something to think about for the next class

- What is the impact of Artificial intelligence on software development using design patterns?

# Thank you!

Prof. Dr. Rosana T. Vaccare Braga | [www.linkedin.com/in/rosana-braga-13778912](https://www.linkedin.com/in/rosana-braga-13778912)

